


ESCOLA POLITÉCNICA DA USP

MATEUS MATZKIN GURZA VINICIUS BARILE LORA FRANCO

# FRAMEWORK DE METAMODELAGEM EXPLICÁVEL PARA ANÁLISE PARAMÉTRICA DE SISTEMAS DE EVENTOS DISCRETOS

São Paulo

2026

ESCOLA POLITÉCNICA DA USP

MATEUS MATZKIN GURZA VINICIUS BARILE LORA FRANCO

# FRAMEWORK DE METAMODELAGEM EXPLICÁVEL PARA ANÁLISE PARAMÉTRICA DE SISTEMAS DE EVENTOS DISCRETOS

Trabalho apresentado ao Departamento de Engenharia Mecatrônica e de Sistemas Mecânicos da Escola Politécnica da USP, como parte das exigências para a conclusão do curso de Engenharia Mecatrônica.

Orientador: Prof. Dr. Fabrício Junqueira

São Paulo

2026

## RESUMO

A Simulação de Eventos Discretos é utilizada na modelagem de sistemas compostos de processos operacionais dinâmicos. A otimização desses sistemas por meio do teste massivo de combinações de parâmetros de entrada possui um custo computacional proibitivo e uma natureza opaca, que oculta a influência exata das variáveis de entrada nas métricas de saída. Para mitigar estes desafios, propõe-se um *framework* em Python que consome dados do *software* de simulação ProModel, para treinar e validar Metamodelos (*Surrogate Models*) capazes de aproximar as respostas do simulador com latência inferior à da simulação tradicional. Adicionalmente, uma camada de explicabilidade aplicada ao Metamodelo, quantifica a sensibilidade do sistema com relação a cada variável de entrada. O *framework* busca facilitar a tomada de decisões de forma mais rápida e informada do que se fossem baseadas na simulação exaustiva.



**Palavras-chave**: Simulação de Eventos Discretos, SED, Metamodelagem, Surrogate Models, Explicabilidade, XAI, ProModel, Projeto de Experimentos, DoE.

## ABSTRACT

Discrete Event Simulation is used in the modeling of systems composed of dynamic operational processes. The optimization of these systems through massive testing of input parameter combinations has a prohibitive computational cost and an opaque nature, which hides the exact influence of input variables on output metrics. To mitigate these challenges, a Python *framework* is proposed that consumes data from the ProModel simulation software to train and validate Metamodels (*Surrogate Models*) capable of approximating simulator responses with lower latency than traditional simulation. Additionally, an explainability layer applied to the Metamodel quantifies the system's sensitivity with respect to each input variable. The *framework* aims to facilitate faster and more informed decision-making than if based on exhaustive simulation.



**Keywords**: Discrete Event Simulation, DES, Metamodeling, Surrogate Models, Explainability, XAI, ProModel, Design of Experiments, DoE.

# LISTA DE ABREVIAÇÕES

| Sigla | Termo | Descrição |
|---|---|---|
| **ANN** | *Artificial Neural Network* (*Rede Neural Artificial*) | Arquitetura de metamodelo baseada em aprendizado de máquina com alta capacidade de aproximar funções não-lineares complexas, ao custo de maior exigência de dados para treinamento e baixa interpretabilidade intrínseca (GOODFELLOW; BENGIO; COURVILLE, 2016; SAVES et al., 2024). |
| **CFD** | *Computational Fluid Dynamics* (*Dinâmica dos Fluidos Computacional*) | Simulação numérica de escoamentos de fluidos. Citada, junto à FEA, como domínio de alto custo computacional onde a Metamodelagem é recorrentemente aplicada, em analogia à sua aplicação em SED (FORRESTER; SÓBESTER; KEANE, 2008). |
| **DoE** | *Design of Experiments* (*Projeto de Experimentos*) | Conjunto de técnicas estatísticas para planejar quais combinações de parâmetros de entrada serão simuladas, garantindo cobertura representativa do espaço paramétrico com o menor número de avaliações possível (MCKAY; BECKMAN; CONOVER, 1979; SALTELLI et al., 2015). |
| **EGO** | *Efficient Global Optimization* (*Otimização Global Eficiente*) | Algoritmo de otimização baseado em Processos Gaussianos (Kriging) utilizado como *benchmark* comparativo na literatura de Metamodelagem para SED (FORRESTER; SÓBESTER; KEANE, 2008; MONTEVECHI et al., 2023). |
| **FEA** | *Finite Element Analysis* (*Análise por Elementos Finitos*) | Método numérico para simulação de fenômenos físicos estruturais. Citado, junto à CFD, como domínio de alto custo computacional onde metamodelos são recorrentemente aplicados, motivando sua adoção em SED (FORRESTER; SÓBESTER; KEANE, 2008). |
| **GBT** | *Gradient Boosted Trees* (*Árvores com Gradiente Impulsionado*) | Método de *ensemble* baseado em árvores de decisão com treinamento sequencial por gradiente, empregado como metamodelo de referência por Montevechi et al. no contexto de SED (MONTEVECHI et al., 2023). |

| | | |
|---|---|---|
| **GP** | *Gaussian Process* (*Processo Gaussiano* / *Kriging*) | Arquitetura de metamodelo que produz predições suaves com estimativa de incerteza probabilística nativa, cujo custo de treinamento cresce cubicamente com o número de amostras, limitando sua escalabilidade (FORRESTER; SÓBESTER; KEANE, 2008; SAVES et al., 2024). |
| **ICE** | *Individual Conditional Expectations* (*Expectativas Condicionais Individuais*) | Técnica de XAI *post-hoc* que exibe como a predição do metamodelo varia para cada observação individual em função de uma variável de entrada, revelando heterogeneidades que a média do PDP poderia ocultar (NAGAHISARCHOGHAEI et al., 2023). |
| **LHS** | *Latin Hypercube Sampling* (*Amostragem por Hipercubo Latino*) | Método de amostragem utilizado no DoE que estratifica o espaço de cada variável em intervalos de igual probabilidade, garantindo distribuição uniforme dos pontos e reduzindo a variância das estimativas do metamodelo (MCKAY; BECKMAN; CONOVER, 1979; VIANA, 2016). |
| **LIME** | *Local Interpretable Model-Agnostic Explanations* (*Explicações Locais Interpretáveis Agnósticas ao Modelo*) | Método de XAI *post-hoc* local que aproxima o comportamento do modelo por um modelo linear simples ao redor de cada predição individual; referenciado como alternativa ao SHAP, com fidelidade inferior em estudos de *benchmarking* (BODRIA et al., 2021; RIBEIRO; SINGH; GUESTRIN, 2016). |
| **MAE** | *Mean Absolute Error* (*Erro Médio Absoluto*) | Métrica de validação de metamodelos que mede o desvio médio entre predição e valor real em termos absolutos, sem penalizar erros grandes de forma desproporcional (CHAI; DRAXLER, 2014). |
| **ML** | *Machine Learning* (*Aprendizado de Máquina*) | Campo da inteligência artificial voltado ao desenvolvimento de algoritmos que aprendem padrões a partir de dados. Base metodológica dos metamodelos treinados no *framework* (SAVES, 2025). |
| **NFL** | *No Free Lunch* (*Princípio da Inexistência de Solução Universal*) | Princípio que estabelece que nenhum metamodelo é universalmente superior em todos os domínios de problema, motivando a comparação sistemática entre múltiplas arquiteturas no *framework* (SAVES, 2025). |
| **OOB** | *Out-of-Bag* (*Fora do Saco*) | Subconjunto de amostras não utilizadas no treinamento de cada árvore em métodos de *ensemble* com *bagging*, empregado como conjunto de validação interna sem necessidade de separação explícita de dados (BREIMAN, 2001). |

| | | |
|---|---|---|
| **PDP** | *Partial Dependence Plots* (*Gráficos de Dependência Parcial*) | Ferramenta de XAI *post-hoc* global que representa a relação média entre uma variável de entrada e a saída do metamodelo, marginalizando o efeito das demais variáveis (NAGAHISARCHOGHAEI et al., 2023). |
| **$R^2$** | *Coefficient of Determination* (*Coeficiente de Determinação*) | Métrica de validação que indica a proporção da variância da saída explicada pelo metamodelo, variando de 0 a 1; utilizada em conjunto com RMSE e MAE para avaliar a fidelidade global do modelo substituto ao simulador (CHAI; DRAXLER, 2014). |
| **RMSE** | *Root Mean Square Error* (*Raiz do Erro Quadrático Médio*) | Métrica de validação de metamodelos que penaliza erros grandes de forma quadrática, sendo sensível a valores atípicos (*outliers*); utilizada em análises comparativas (*benchmarking*) junto a MAE e $R^2$ (CHAI; DRAXLER, 2014). |
| **SED** | *Discrete Event Systems* (*Sistemas a Eventos Discretos*) | Classe de sistemas cujo estado evolui de forma instantânea em resposta a eventos discretos no tempo. Contexto central do trabalho, no qual o ProModel é o simulador de referência (SCHRIBER; BRUNNER, 2003). |
| **SHAP** | *Shapley Additive Explanations* (*Explicações Aditivas de Shapley*) | Método de XAI *post-hoc* fundamentado na teoria dos jogos cooperativos que atribui a cada variável de entrada sua contribuição marginal na predição do metamodelo, de forma globalmente consistente e com suporte empírico superior ao LIME em dados tabulares (NAGAHISARCHOGHAEI et al., 2023; BODRIA et al., 2021). |
| **XAI** | *eXplainable Artificial Intelligence* (*Inteligência Artificial Explicável*) | Conjunto de técnicas e metodologias que tornam as predições de metamodelos de ML interpretáveis e auditáveis por humanos. No *framework*, é a camada que resolve o problema de opacidade do simulador ao quantificar a contribuição marginal de cada parâmetro de entrada nos indicadores de desempenho (SAVES, 2025; NAGAHISARCHOGHAEI et al., 2023). |

# Lista de Figuras


Figura 1 Diagrama simplificado das conexões e fluxo do sistema. . . . . 32
Figura 2 Diagrama do ciclo de desenvolvimento do *framework*. . . . . 37
Figura 3 Cenário 1 - linha sequencial com três processos em série. . . . . 39
Figura 4 Cenário 2 - roteamento probabilístico para três caminhos de comprimento distinto. . . . . 39
Figura 5 Cenário 3 - sistema multi-produto com sequências de processos distintas por tipo de produto. . . . . 40
Figura 6 Comparação ilustrativa entre os métodos de amostragem no hipercubo unitário. . . . . 51
Figura 7 Planilha de execução em lote - cada linha corresponde a um cenário simulado. . . . . 56
Figura 8 Diagrama de Gantt - Macroetapas do TCC 2026 . . . . . 61
Figura 9 Barra de ferramentas do ProModel. . . . . 68
Figura 10 Cockpit de entradas e saídas do Cenário 1. . . . . 70


# Lista de Tabelas


Tabela 1 Comparação dos cenários sintéticos desenvolvidos no ProModel. . . . . 41
Tabela 2 Comparação das técnicas de amostragem consideradas no DoE. . . . . 42
Tabela 3 Comparação qualitativa das famílias de metamodelo consideradas. . . . 43
Tabela 4 Gatilhos de revisão de Metamodelos e Treinamento. . . . . 44
Tabela 5 Métricas de fidelidade preditiva adotadas na avaliação. . . . . 45
Tabela 6 Gatilhos de revisão da Avaliação Estatística do Desempenho. . . . . 46
Tabela 7 Técnicas de explicabilidade aplicadas e a pergunta que cada uma responde. 46
Tabela 8 Metodologia de Validação Teórica por tipo de caso. . . . . 47
Tabela 9 Formatos de troca entre os blocos do *framework*. . . . . 63
Tabela 10 Funções do módulo de Design of Experiments. . . . . 64
Tabela 11 Funções do módulo de replicação. . . . . 65
Tabela 12 Funções do módulo de interface com o ProModel. . . . . 65
Tabela 13 Funções do módulo de pré-processamento. . . . . 66
Tabela 14 Variantes disponíveis em cada módulo de metamodelo. . . . . 67

# Sumário

**1 INTRODUÇÃO . . . . . 10**
1.1 Formalização do Problema Abordado . . . . . 12
1.2 Motivação . . . . . 13
1.2.1 Interesse Econômico e Social . . . . . 13
1.2.2 Mérito Técnico-Científico . . . . . 14
1.3 Objetivos . . . . . 14
1.3.1 Produtos Esperados . . . . . 14
1.3.2 Testes para Validação . . . . . 15
1.4 Estrutura da Monografia . . . . . 15

**2 REVISÃO BIBLIOGRÁFICA . . . . . 17**
2.1 Fundamentação Teórica . . . . . 17
2.1.1 Simulação de Eventos Discretos . . . . . 17
2.1.2 Projeto de Experimentos (DoE) . . . . . 18
2.1.3 Metamodelagem . . . . . 21
2.1.4 Explicabilidade . . . . . 24
2.2 Estado da Arte . . . . . 26
2.2.1 Sinergia entre Metamodelagem e XAI . . . . . 27
2.2.2 Projetos Similares . . . . . 28
2.3 Conclusões Parciais . . . . . 29

**3 PLANO DE PROJETO . . . . . 31**
3.1 Resumo da Abordagem . . . . . 31
3.2 Requisitos . . . . . 32
3.2.1 Requisitos Funcionais . . . . . 32
3.2.2 Requisitos Não-Funcionais . . . . . 34
3.3 Metodologia . . . . . 35
3.3.1 Ciclo de Desenvolvimento . . . . . 36
3.3.2 Modelo de Sistema de Eventos Discretos . . . . . 38
3.3.3 Projeto de Experimentos (DoE) . . . . . 41
3.3.4 Simulador e Aplicação no Espaço Amostral . . . . . 42
3.3.5 Metamodelos e Treinamento . . . . . 43

3.3.6 Testes e Plano de Avaliação Estatística . . . . . 44
3.3.7 Camada de Explicabilidade . . . . . 46
3.3.8 Plano de Validação Teórica . . . . . 47

**4 IMPLEMENTAÇÃO . . . . . 48**
4.1 Arquitetura do projeto . . . . . 48
4.2 Geração do plano amostral . . . . . 50
4.3 Tratamento da estocasticidade por replicação . . . . . 51
4.4 Interface com o simulador . . . . . 52
4.5 ProModel . . . . . 52
4.5.1 Implementação do Cenário 1 - Linha Sequencial Simples . . . . . 52
4.5.2 Implementação do Cenário 2 - Roteamento Probabilístico . . . . . 54
4.5.3 Implementação do Cenário 3 - Sistema Multi-Produto . . . . . 54
4.5.4 *Cockpit*, Execução em Lote e Preparação de Dados para o DoE . . . . . 55
4.5.5 Verificação dos Modelos Implementados . . . . . 57
4.6 Pré-processamento . . . . . 58
4.7 Famílias de metamodelos . . . . . 59
4.8 Avaliação estatística . . . . . 60
4.9 Camada de explicabilidade . . . . . 60
4.10 Validação teórica . . . . . 60

**Apêndice A Relatório de Acompanhamento do TCC . . . . . 61**

**Apêndice B Organização do repositório . . . . . 62**

**Apêndice C Módulos e funções . . . . . 64**

**Apêndice D Funcionamento ProModel e Modelos . . . . . 68**

**REFERÊNCIAS BIBLIOGRÁFICAS . . . . . 71**

# 1 INTRODUÇÃO

A modelagem computacional de sistemas ocupa um lugar consolidado na prática de engenharia porque substitui a necessidade da análise física do sistema, na qual é necessário executar o processo para estudá-lo (SAVES, 2025). A análise por experimentação do sistema real costuma ser proibitiva em larga escala tendo em vista as exigências de custo e tempo associadas a ela. Em contrapartida, a utilização de métodos de simulação computacional, nesse contexto, é aplicada a diversas áreas da engenharia porque resolve os gargalos citados do processo de experimentação prática. Com o uso de *softwares* adequados, tais métodos permitem reproduzir problemas com pouco esforço, sendo possível alterar e observar os efeitos de diversos parâmetros, facilitando a visualização e predição de fenômenos.

No entanto, de acordo com Saves (2025), *softwares* de simulação ainda apresentam restrições práticas do ponto de vista de custo computacional, opacidade e credibilidade. Primeiro, com relação ao custo computacional, entende-se que o processo de simulação ainda é moroso ao passo que exige a execução completa de passagens análogas àquelas executadas no processo real para então retornar um resultado. Este resultado corresponde à saída referente a uma única combinação de parâmetros de entrada no modelo da simulação. A necessidade de executar combinações de parâmetros inicias em massa, relacionada ao processo de otimização e exploração de configurações de sistemas, instigou o questionamento acerca do custo computacional da utilização de simulações e o tempo que estas exigem para a realização desse tipo de tarefa. Segundo, a questão da opacidade e credibilidade esta relacionada com a dificuldade de interpretar e validar os mecanismos que geram as saídas da simulação. Para o *software* de simulação ProModel por exemplo, o programa integrado de otimização SimRunner, é capaz de retornar a melhor escolha de parâmetros de entrada em função do resultado obtido nas variáveis de saída, mas não explicar porque a determinada configuração é superior, ou seja, não atribui valor à participação das entradas no resultado obtido. Estrategicamente, este problema de explicabilidade é relevante do ponto de vista da comunicação com *stakeholders* e decisores. Inserida nesse cenário, a pesquisa por métodos alternativos passou a buscar opções que não dependessem da execução das passagens sequenciais do processo simulado para a obtenção de resultados e, concomitantemente, soluções para o problema da explicabilidade e técnicas de suporte à decisão.

Como solução para o problema do custo computacional, segundo (SAVES, 2025), um método encontrado, cuja utilização vem sendo abordada na literatura do campo de Sistemas a Eventos Discretos (SED) atualmente, foi o da Metamodelagem. Também chamada de *Surrogate Modelling*, esta metodologia explora a intersecção entre métodos de *Machine Learning* para criar modelos com

custo computacional reduzido para a pesquisa no espaço de parâmetros de um sistema. A elaboração desses metamodelos parte de um conjunto de dados de entrada e saída gerado pelo simulador de referência para treinar modelos substitutos capazes de aproximar o comportamento do simulador com latência significativamente inferior à da simulação tradicional (SAVES et al., 2024). Uma vez treinados, esses modelos permitem a exploração massiva do espaço paramétrico sem a necessidade de executar novas simulações completas, viabilizando processos de otimização e análise de cenários antes computacionalmente proibitivos (MONTEVECHI et al., 2023).

Já com relação ao problema de opacidade e credibilidade, o tema de *eXplainable Artificial Intelligence (XAI)* se destaca na literatura por sua sinergia com os métodos de *Machine Learning* que complementam à abordagem de *Surrogate Modeling*. A questão da explicabilidade é destacada no trabalho (RIBEIRO; SINGH; GUESTRIN, 2016) onde é levantada a importância da capacidade de entender o comportamento dos modelos ao invés de aceitá-los como caixa-preta (*black box*). As técnicas de XAI se dividem, de forma geral, entre métodos intrinsecamente interpretáveis - como árvores de decisão e modelos lineares - e métodos *post-hoc*, aplicados sobre modelos já treinados para gerar explicações a posteriori (PATIL; PATIL, 2023). Entre os métodos *post-hoc* mais consolidados destacam-se o SHAP, o PDP e o ICE, ferramentas que quantificam a contribuição marginal de cada variável de entrada nas predições do modelo (NAGAHISARCHOGHAEI et al., 2023). A evolução dessas ferramentas possibilitou a conexão com estruturas de Metamodelagem. Neste contexto, a aplicação de XAI surgiu como forma de abordar as dores dos *stakeholders* e decisores e facilitar a compreensão da importância dos parâmetros de um sistema metamodelado.

A convergência dessas duas frentes, envolvendo a Metamodelagem como substituto computacionalmente eficiente do simulador, e o XAI como camada de interpretabilidade sobre os modelos treinados, configura uma abordagem promissora para superar as limitações identificadas na literatura acerca da simulação tradicional (SAVES, 2025). O presente trabalho explora essa sinergia aplicada ao contexto de Sistemas de Eventos Discretos e ancora a teoria à realidade por meio da implementação prática ligada a uma ferramenta recorrentemente utilizada no mercado (ProModel), assim delimitando o escopo como especificado na seção 1.1. Resumidamente, o trabalho proposto visa explorar a fronteira da análise e modelagem eficiente de sistemas por meio de um *framework* que, na prática, sirva de subsídio para a implementação desse tipo de abordagem na industria, sendo essa aplicação uma lacuna ainda pouco explorada.

## 1.1 Formalização do Problema Abordado

Proposta a situação atual no campo das simulações e modelagem computacional, a exploração do espaço paramétrico por meio de simulação tradicional é computacionalmente custosa (SAVES, 2025): cada simulação pode demandar minutos a horas de processamento, tornando a análise exaustiva de cenários um gargálo durante a otimização de sistemas e gerando atrasos em processos decisórios na industria.

Mesmo quando resultados de simulação estão disponíveis, a relação entre variáveis de entrada (como taxas de chegada, capacidades de recursos, políticas de escalonamento e tempos de processos) e os indicadores de desempenho de saída (como *throughput*, ocupação, filas e tempo em operação) permanece opaca (SAVES, 2025). Isso complexifica a identificação das intervenções que efetivamente tornam o processo mais eficiente, bem como dificulta discernir qual a proporção com a qual cada ajuste contribui para a melhoria do desempenho do sistema, prejudicando a capacidade de priorização dos decisores envolvidos ao obscurecer quais modificações são de fato essenciais dentre aquelas identificadas em cenários com variação extensa dos parâmetros de entrada.

No cenário de simulação tradicional, ferramentas de Simulação de SED, como explicadas em (SCHRIBER; BRUNNER, 2003), lidam com modelos que variam o seu estado de forma pontual e imediata em resposta a eventos discretos no tempo, que seriam ocorrências marcadas por instantantaneidade e associadas unidades de tempo internas ao modelo, evitando a complexidade de abordagens dinâmicas em tempo contínuo. No campo de SED, um *software* acessível, utilizado com frequência na modelagem de sistemas produtivos e logísticos complexos é o ProModel. Segundo (BENSON, 1997), é um *software* prático no qual é fácil aprender a desenvolver modelos complexos que simulam com grande acurácia processos reais, através de todas as funcionalidades disponíveis no sistema.

O ProModel permite o desenvolvimento de modelos com controle de parâmetros de entrada por meio de planilhas do *Excel*, elaboração de cenários de simulação e exportação dos indicadores de desempenho resultantes em formato tabular (BENSON, 1997). Apesar de sua relevância na industria, o *software*, como outros simuladores, é sujeito às mesmas limitações de custo computacional e opacidade discutidas anteriormente (SAVES, 2025), representando um candidato ao processo de Metamodelagem. Além disso, o ProModel possui integração nativa com planilhas, o que confere uma vantagem operacional relevante para o presente trabalho: ele viabiliza a parametrização sistemática das entradas e a coleta estruturada das saídas, etapas essenciais à execução de um Projeto de Experimentos (DoE) para composição dos conjuntos de dados necessários no treinamento dos metamodelos. Essa característica e as demais citadas anteriormente tornam o ProModel uma escolha

pertinente como simulador de referência no *framework* proposto, ao passo que permite uma implementação modular, integrada e reprodutível do processo de amostragem, treinamento de metamodelos e aplicação de explicabilidade, tudo isso enquanto simultaneamente representa uma ferramenta relevante em aplicações reais no mercado.

Sintetizando os desafios expostos e recapitulando a proposta de trabalho, o presente projeto busca articular a aplicação industrial da simulação de eventos discretos e a fronteira de Metamodelagem e Inteligência Artificial Explicável (XAI) unidas, sinergia representativa do estado da arte da análise otimizada de sistemas. O que se pretende explorar é a viabilidade e os caminhos práticos para integrar essas frentes em uma cadeia analítica estruturada. Ao utilizar o ProModel como ambiente gerador de dados, o trabalho busca investigar como essa orquestração de procedimentos bem definidos pode ajudar a mitigar, simultaneamente, o custo computacional e a opacidade associadas a um produto de presença consolidada no mercado, oferecendo um referencial metodológico para apoiar a tomada de decisão em ambientes onde a velocidade do processo decisório e sua interpretabilidade tem alto impacto.

## 1.2 Motivação

A motivação para o desenvolvimento deste trabalho tem origem em duas linhas de raciocínio: uma voltada para a perspectivas de impacto em aspectos econômicos e sociais, como a aceleração de decisões estratégicas e sua justificabilidade por meio de critérios quantitativos; e outra para a perspectiva técnico-científica, e a relevância acadêmica do desenvolvimento de processos claros para a integração produtiva da Iteligência Artificial em projetos de engenharia.

### 1.2.1 Interesse Econômico e Social

A tomada de decisão baseada em dados em ambientes produtivos e logísticos é um dos pilares da Indústria 4.0. Empresas que utilizam simulação para dimensionar operações - centros de distribuição, linhas de manufatura ou terminais portuários - enfrentam o gargalo do custo computacional de exploração paramétrica, análises que poderiam orientar decisões de investimento de grande porte ficam represadas por limitações de tempo e processamento (SAVES, 2025).

O metamodelo proposto reduz a latência de inferência, ampliando o acesso a análises de cenário durante reuniões de planejamento operacional. A camada XAI substitui julgamentos sem embasamento, ou embasados por critérios subjetivos, por justificativas quantitativas e visuais, aumentando a rastreabilidade das decisões. Simultaneamente, a capacidade de quantificar e comparar a contribuição

de parâmetros de entrada nos resultados do sistema permite abordagens incisivas e priorizadas, com possíveis impactos na redução de gastos de implementação das soluções de melhoria do sistema real.

### 1.2.2 Mérito Técnico-Científico

Do ponto de vista da Engenharia Mecatrônica, o trabalho integra três áreas de relevância contemporânea:

- **Modelagem e Simulação:** Aprofundamento em SED e suas limitações de escalabilidade, tema central em disciplinas de sistemas de produção e controle;
- **Aprendizado de Máquina Aplicado a Sistemas:** Uso de técnicas de ML como substitutos analíticos de modelos físicos complexos - abordagem evidenciada na literatura de *surrogate modeling* (SAVES et al., 2024) para simulações de alto custo computacional (e.g., FEA, CFD e SED);
- **Inteligência Artificial Explicável (XAI):** Área de crescente importância em aplicações de engenharia, onde a interpretabilidade dos modelos é requisito para adoção em ambientes de alto impacto.

O diferencial científico reside na integração dessas três frentes em um *framework* validado empiricamente com aplicação prática em um simulador estabelecido no mercado (ProModel), contribuindo para a literatura de Metamodelagem aplicada a SED com foco em explicabilidade.

## 1.3 Objetivos

O objetivo deste trabalho é projetar e implementar uma ferramenta facilmente replicável para auxiliar no processo de exploração de configurações de sistema, seja a configuração o conjunto de parâmetros que definem o modelo. Almeja-se, nesse sentido, tornar mais rápida a obtenção dos resultados esperados para um dado conjunto de parâmetros e facilitar a busca pelas configurações ótimas do sistema, resolvendo o problema do custo computacional e diminuindo a latência de resposta, questões exploradas na seção 1.1. Simultaneamente, tem-se como objetivo facilitar a compreensão dos efeitos de mudanças dos parâmetros nas variáveis de saída, possibilitando decisões informadas e analises robustas acerca das possíveis configurações do sistema.

### 1.3.1 Produtos Esperados

Os produtos a serem construídos para cumprir com o objetivo deste trabalho são:

1. Uma **cadeia de processamento em Python** completa, documentada e reutilizável para geração de dados via DoE (*Design of Experiments*) no ProModel, treinamento e seleção de metamodelos e aplicação da camada de explicabilidade;

2. A **comparação sistemática** de arquiteturas de metamodelagem (*Random Forest*, ANN e outros algoritmos relevantes da literatura) em termos de acurácia, generalização, tempo de treinamento, latência de inferência e custo computacional;

3. Um conjunto de **métricas de fidelidade** que quantificam o grau de aderência do metamodelo ao simulador de referência, incluindo análise de verossimilhança e *benchmarks* de fidelidade;

4. **Relatórios de explicabilidade** que correlacionem as importâncias de variáveis de entrada extraídas via camada de explicabilidade com a teoria de modelos analíticos de sistemas produtivos e logísticos, validando a coerência do metamodelo;

5. ***Framework* aplicado** de forma funcional em pelo menos dois sistemas: modelos sintéticos de referência e um caso industrial real (dados anonimizados).

### 1.3.2 Testes para Validação

Ao final do trabalho, os seguintes resultados metrificáveis serão utilizados para validar se o objetivo foi cumprido utilizando os produtos esperados:

1. **Taxa de acerto das variáveis de saída:** Análise quantitativa das predições corretas do Metamodelo usando métricas clássicas de *Machine Learning* ($R^2$, RMSE, MAE);

2. **Taxa de acerto da configuração ótima:** Análise quantitativa da capacidade de identificar a melhor configuração entre as possibilidades;

3. **Tempo de inferência:** Comparação quantitativa entre o tempo decorrido na execução da mesma tarefa por meio do Metamodelo e da simulação tradicional;

4. **Testes de explicabilidade:** Confirmação do efeito quantificado de cada variável provindo da camada de XAI por meio da replicação no *software* ProModel.

## 1.4 Estrutura da Monografia

Esta monografia compila o estudo, planejamento e projeto elaborados antes e durante a construção do *Framework* de Metamodelagem Explicável para Análise Paramétrica de Sistemas de Eventos Discretos. A divisão do conteúdo segue a estrutura exposta a seguir:

- **Introdução**: Seção abordando o panorama geral do problema explorado; a lacuna específica abordada ao longo do trabalho; a motivação por trás do projeto; e os objetivos e resultados almejados;

- **Revisão Bibliográfica**: Aqui são tratados os conceitos relevantes para o projeto e o estado da arte encontrado na literatura. Esta seção contém a fundamentação teórica para as decisões tomadas ao longo do projeto;

- **Plano de Projeto**: São discutidos nesta seção as questões relacionadas à definição de requisitos de projeto e explicadas as abordagens em cada uma das etapas relevantes para a construção do *framework*;

- **Implementação**: Explica-se nesta seção a forma como foram implementadas as diversas etapas do *framework*, esclarecend como o Plano de Projeto foi materializado e aplicado na prática;

- **Resultados**: São apresentados nesta seção os resultados obtidos com a aplicação do *framework* em casos de estudo explicados ao longo da monografia.

# 2 REVISÃO BIBLIOGRÁFICA

Com o objetivo de estabelecer a base teórica necessária para a execução do trabalho e contextualizar aonde este projeto se insere no panorama da literatura relevante, esta seção traz duas frentes de conteúdo. A primeira, subseção 2.1 trata dos principais conceitos já e campos articulados no trabalho. Esses fundamentos são, de forma geral, áreas de pesquisa já bem conhecidas, cuja integração no *framework* permite abordar uma fronteira menos explorada. Essa fronteira é abordada na subseção seguinte (2.2).

## 2.1 Fundamentação Teórica

A fundamentação teórica trazida contextualiza os principais conceitos relevantes para o posterior entendimento do trabalho e do Estado da Arte (seção 2.2). Estes são: Simulação de Eventos Discretos, Projeto de Experimentos, Metamodelagem e Explicabilidade. Cada um desses campos citados é, individualmente, um tema explorado e estabelecido na literatura. Por outro lado, quando articulados em união, esses blocos de conhecimento formam a fronteira do conhecimento abordado neste trabalho.

### 2.1.1 Simulação de Eventos Discretos

A Simulação a Eventos Discretos (SED) é uma metodologia consolidada para a modelagem e análise de sistemas cujo estado evolui de forma instantânea em resposta a eventos discretos no tempo. Conforme definido por Schriber e Brunner (2003), um modelo SED representa o sistema como uma sequência de eventos que alteram variáveis de estado em instantes específicos, com o tempo avançando de evento em evento, sem variação contínua entre eles. Desde sua origem nos anos 1950, a SED tem sido empregada com êxito em domínios como manufatura, logística, saúde e sistemas de serviços, onde a dinâmica operacional pode ser descrita por entidades que transitam entre recursos ao longo do tempo (SCHRIBER; BRUNNER, 2003).

No contexto industrial, a SED se destaca por permitir que pesquisadores e gestores analisem o desempenho de um sistema sob diferentes condições operacionais sem a necessidade de intervenção no processo real, eliminando os riscos e custos associados à experimentação física (SCHRIBER; BRUNNER, 2003). Essa capacidade é especialmente relevante em sistemas produtivos e logísticos complexos, onde a interdependência entre recursos, filas e políticas de escalonamento impede uma abordagem analítica. O ProModel é um exemplo de *software* de SED amplamente utilizado nesse

contexto: segundo Benson (1997), sua interface orientada a processos e a integração nativa com planilhas do Excel o tornam acessível para o desenvolvimento de modelos de alta complexidade, com controle paramétrico de entradas e exportação estruturada de indicadores de desempenho.

Apesar de sua consolidação, SED apresenta limitações práticas que restringem seu uso em processos de otimização, os quais exigem a simulação exaustiva de combinações de parâmetros de entrada. Saves (2025) identifica duas restrições centrais. A primeira é o custo computacional: cada execução do simulador reproduz sequencialmente a lógica interna do sistema modelado, percorrendo todas as interações entre entidades, recursos e eventos. Esse processo retorna o resultado referente a uma única combinação de parâmetros de entrada, de modo que a avaliação de múltiplos cenários exige executar o simulador repetidamente, com tempo de processamento que pode variar de minutos a horas por execução. A segunda restrição é a opacidade: o simulador produz saídas observáveis, mas não esclarece ou quantifica a contribuição de cada parâmetro de entrada no resultado final. Ferramentas como o SimRunner, módulo de otimização integrado ao ProModel, são capazes de identificar configurações de parâmetros que maximizam um indicador de desempenho, mas não explicam por que determinada configuração é superior às demais (BENSON, 1997; SAVES, 2025).

Um desafio adicional específico de SED é o caráter estocástico dos modelos. Ao contrário de simulações determinísticas, execuções distintas do mesmo modelo SED com os mesmos parâmetros de entrada produzem saídas diferentes em função de aleatoriedades internas ao sistema, que conduzem eventos como tempos de chegada, duração de processos e falhas de equipamentos (MONTEVECHI et al., 2023). Isso significa que uma única execução por configuração paramétrica de entrada é insuficiente para obter o comportamento esperado do sistema naquele ponto, sendo necessária a realização de múltiplas replicações para que a média das saídas seja estatisticamente representativa. Esse requisito amplifica diretamente o custo computacional da exploração paramétrica, pois cada ponto avaliado no espaço de parâmetros demanda não uma, mas várias execuções do simulador (BINOIS; GRAMACY; LUDKOVSKI, 2019; MONTEVECHI et al., 2023). É sobre essas duas limitações (custo computacional e opacidade) que as frentes de Metamodelagem e XAI, respectivamente, atuam de forma complementar, conforme detalhado nas subseções seguintes.

### 2.1.2 Projeto de Experimentos (DoE)

O *Design of Experiments* (DoE), ou Projeto de Experimentos, estabelece-se como uma abordagem metodológica e estatística para o planejamento e execução da amostragem de dados, sendo a amostragem uma forma de selecionar um conjunto reduzido de observações capazes de representar a totalidade dos dados. Originado a partir dos trabalhos de Ronald Fisher (FISHER, 1935), o DoE

se diferencia de métodos que variam isoladamente uma variável por vez ao propor a manipulação simultânea e estruturada de múltiplas variáveis por observação. Segundo Montgomery (2017), essa estruturação permite que as relações de causa e efeito entre os parâmetros sejam compreendidas de forma eficiente a partir de um conjunto finito e previamente definido de observações.

A importância da aplicação do DoE reside na otimização da alocação de recursos durante a exploração de um dado fenômeno, característica que se torna crítica em ambientes onde a experimentação ou obtenção de dados possui elevado custo, como no caso de simulações computacionais (FORRESTER; SÓBESTER; KEANE, 2008). Um plano experimental bem delineado assegura uma cobertura espacial representativa do domínio paramétrico, quantificando não apenas os efeitos isolados de cada fator, mas também as interações estocásticas complexas entre eles (MONTGOMERY, 2017). Nesse cenário, técnicas formais de amostragem e planejamento do domínio (MCKAY; BECKMAN; CONOVER, 1979) tornam-se ferramentas essenciais para reduzir a variância e mitigar incertezas enquanto simultaneamente reduzindo a carga de trabalho computacional ao passo que delimita um conjunto menor de observações a serem processadas.

No contexto de Metamodelagem, o DoE participa como uma etapa inicial do processo, precedendo o treinamento dos metamodelos em si. Explicada na seção 2.1.3, essa participação se dá como uma etapa de *design* do espaço amostral no qual o modelo substituto deverá ser treinado e, portanto, trata diretamente de uma das principais utilidades de um metamodelo: a capacidade de representar um espaço denso de respostas a diferentes configurações paramétricas de entrada a partir de uma quantidade limitada de dados, permitindo que uma análise seja feita sobre os resultados de um simulador exigindo apenas executar uma fração das simulações para treinar o substituto (FORRESTER; SÓBESTER; KEANE, 2008; MCKAY; BECKMAN; CONOVER, 1979). Um metamodelo treinado sobre a totalidade do espaço amostral é inútil do ponto de vista da eficiência computacional, ao passo que exigiria simular todos os cenários na latência original do simulador.

O DoE deve levar em consideração características do tipo de simulação sendo abordada. No caso de SED, como explicado na seção 2.1.1, a estocasticidade interna dos processos indica que uma única execução por configuração paramétrica não é suficiente para caracterizar o resultado esperado dessa configuração. A literatura aponta que, nesse contexto, o DoE enfrenta um dilema fundamental entre replicação e exploração (BINOIS; GRAMACY; LUDKOVSKI, 2019): replicar um ponto amostral, ou seja, executar o simulador múltiplas vezes com os mesmos parâmetros de entrada tal que as respostas variem segundo eventos probabilísticos, permite estimar a média e a variância da resposta naquele ponto, separando o sinal real do ruído estocástico; explorar um novo ponto, por outro lado, amplia a cobertura do espaço paramétrico e melhora a capacidade de generalização do metamodelo

para regiões não observadas. O custo computacional total é fixo: cada replicação adicional consome o orçamento de simulações que poderia ser destinado à exploração de uma nova configuração, e vice-versa (BINOIS; GRAMACY; LUDKOVSKI, 2019). A decisão acerca do número de replicações por ponto deve, portanto, equilibrar a necessidade de estimativas estatisticamente confiáveis das saídas com a cobertura representativa do espaço de parâmetros. Na prática, a definição da quantidade de replicações por ponto varia de caso para caso, levando em consideração aspectos como a robustez, do modelo escolhido, a ruído estatístico, o nível de acurácia almejado, e os recursos computacionais disponíveis. Um exemplo é o número adotado por Montevechi et al. (2023), que utilizam 15 replicações por ponto no seu estudo de caso em manufatura.

As diferentes abordagens de DoE disponíveis na literatura variam principalmente em como distribuem os pontos amostrais no espaço paramétrico. A amostragem aleatória simples (*Monte Carlo*) é a abordagem mais básica, mas apresenta convergência lenta e tende a produzir aglomerações (*clusters*) e vazios no espaço de parâmetros, comprometendo a representatividade da amostra (SALTELLI et al., 2015). O *Latin Hypercube Sampling* (LHS), proposto por McKay, Beckman e Conover (1979), resolve esse problema por meio de uma estratificação do espaço de cada variável em intervalos de igual probabilidade, garantindo que cada intervalo seja amostrado exatamente uma vez. O resultado é uma distribuição mais uniforme dos pontos no espaço paramétrico com o mesmo número de avaliações, reduzindo a variância das estimativas do metamodelo (MCKAY; BECKMAN; CONOVER, 1979; VIANA, 2016). Para maior controle sobre a uniformidade, variantes otimizadas do LHS, como o *maximin LHS*, que maximiza a distância mínima entre pontos, produzem *designs* com melhor cobertura espacial (VIANA, 2016). Sequências quasi-aleatórias constituem uma terceira abordagem: são sequências determinísticas[1] de baixa discrepância[2] projetadas para preencher o espaço de forma sistematicamente mais uniforme do que o LHS, apresentando propriedades de convergência superiores especialmente em espaços de alta dimensionalidade (SALTELLI et al., 2015).

Adicionalmente, a literatura distingue estratégias de DoE fixas (*one-shot*) de estratégias incrementais (*infill-search*). Na estratégia fixa, o conjunto amostral completo é definido antes da execução das simulações e o metamodelo é treinado exclusivamente sobre esses dados (MONTEVECHI et al., 2023). Na estratégia incremental, um conjunto inicial é simulado, o metamodelo é treinado e, com base em critérios de aquisição (como a melhoria esperada ou a incerteza de predição) novos pontos são selecionados iterativamente para simulação e incorporados ao conjunto de treinamento (FORRESTER; SÓBESTER; KEANE, 2008; MONTEVECHI et al., 2023). A estratégia incremental tende

[1] Cada número na sequência depende do seu antecessor (ou é gerado por uma regra fixa) sendo, então, reprodutível.

[2] A "discrepância"é uma medida quantitativa da uniformidade de uma sequência. Uma "baixa discrepância"significa que os pontos estão espalhados de forma mais uniforme do que seria esperado com amostragem puramente aleatória

a ser mais eficiente em termos de orçamento de simulações, pois concentra as avaliações nas regiões do espaço paramétrico onde o metamodelo ainda apresenta maior incerteza ou maior potencial de melhoria.

### 2.1.3 Metamodelagem

Saves (2025) define Metamodelos como o "remédio prático para os desafios de custo e escalabilidade"[3] capaz de "aprender aproximações econômicas de modelos custosos de simulação a partir de uma quantidade reduzida de simulações". Como resultado, também indica que metamodelos permitem a exploração exaustiva de cenários, aplicação de processos de otimização de sistemas e integração com outras ferramentas, além de fornecer um "ambiente controlado"e "interrogável", menos sensível a "ruídos"que podem interferir nas respostas de um simulador.

De forma geral, a Metamodelagem não está presa a um paradigma de simulação específico, e pode ser aplicada às famílias de simuladores que "definem o mapeamento de parâmetros de entrada a saídas de interesse"mediante "regras"que definem a transformação de um no outro (SAVES, 2025). A aplicação de metamodelos em simulação emergiu no contexto de modelos baseados em equações, domínio tradicional da análise de elementos finitos (FORRESTER; SÓBESTER; KEANE, 2008), e avançou como metodologia possibilitando lidar com a latência excessiva de simulações, cuja origem pode ser rastreada a algumas diferentes fontes de lentidão no processo de inferência: em modelos baseados em agentes, a estocasticidade dos comportamentos simulados é a causa da lentidão por exigir extensa replicação de cenários para obter resultados estatisticamente relevantes; e, em SED, abordado ao longo deste trabalho, a fonte da alta latência pode ser ligada à execução sequencial de eventos e gerenciamento do processo e recursos, onde a estocasticidade também age, tornando a simulação exaustiva uma tarefa custosa (MONTEVECHI et al., 2023).

A Metamodelagem, hoje, foca na utilização de modelos de aprendizado de máquina e se organiza em torno de dois objetivos centrais e frequentemente antagônicos: maximizar a fidelidade do metamodelo ao simulador de referência e minimizar o custo computacional de treinamento e inferência (FORRESTER; SÓBESTER; KEANE, 2008; SAVES, 2025). De forma geral, o *pipeline* de metamodelagem segue uma cadeia de etapas interdependentes: planejamento do espaço amostral via *Design of Experiments* (DoE), execução das simulações nos pontos planejados, treinamento do metamodelo sobre os dados gerados, validação da fidelidade do modelo treinado e, por fim, sua utilização para exploração paramétrica ou otimização (FORRESTER; SÓBESTER; KEANE, 2008; MONTEVECHI et al., 2023). A literatura aborda os desafios dessa cadeia a partir de duas frentes principais.

[3]Tradução Livre pelos autores

A primeira frente diz respeito à escolha da arquitetura de aprendizado de máquina. Saves (2025) identifica cinco atributos como objetivos concorrentes que orientam a seleção de metamodelos: *accuracy* (acurácia), *smoothness* (suavidade), *interpretability* (interpretabilidade), *fast prediction* (predição rápida) e *uncertainty estimate* (estimativa de incerteza). Segundo o autor, melhorar um desses atributos frequentemente implica sacrificar outro, comportamento formalizado pelo princípio *No Free Lunch* (NFL), que estabelece que nenhum metamodelo é universalmente superior em todos os domínios de problema (SAVES, 2025).

A *accuracy* determina a capacidade do modelo de capturar as relações subjacentes entre entradas e saídas do simulador sem sobreajuste (*overfitting*) nem subajuste (*underfitting*). A *smoothness* refere-se à continuidade da função de resposta produzida pelo modelo: superfícies suaves facilitam a aplicação de métodos de XAI *post-hoc* e são fisicamente coerentes com sistemas cujas saídas variam gradualmente em relação aos parâmetros de entrada (SAVES, 2025). A *interpretability* diz respeito à capacidade de extrair diretamente da estrutura do modelo relações entre variáveis de entrada e saída, sem necessidade de ferramentas externas (MOLNAR, 2025; SAVES, 2025). O atributo de *fast prediction* refere-se à velocidade de inferência do modelo já treinado, relevante especialmente para métodos de XAI que demandam um grande número de avaliações do metamodelo, ou seja exigem múltiplas inferências para fornecer explicações, limitando a velocidade de predição, como o *KernelSHAP*; cabe notar que, para algumas arquiteturas, o cálculo do SHAP[4] pode ser realizado analiticamente, como o *TreeSHAP* para *Random Forests*, eliminando essa restrição (SAVES, 2025). Por fim, a *uncertainty estimate* quantifica a confiança do modelo em cada predição, informação que enriquece a interpretabilidade ao sinalizar regiões do espaço paramétrico onde o metamodelo é menos confiável (SAVES, 2025).

Sob essa perspectiva, as principais famílias de algoritmos apresentam perfis distintos de *tradeoff*. Processos Gaussianos (*Kriging*) destacam-se pela *smoothness* inerente das suas predições e pela capacidade nativa de fornecer estimativas de incerteza probabilísticas, características valiosas em contextos de otimização e análise de confiabilidade (FORRESTER; SÓBESTER; KEANE, 2008; SAVES et al., 2024). Em contrapartida, seu custo de treinamento cresce cubicamente com o número de amostras, o que compromete a escalabilidade em conjuntos de dados maiores (SAVES et al., 2024). Métodos de *ensemble*[5] baseados em árvores de decisão, como *Random Forests*, oferecem alta generalização, robustez ao sobreajuste e elevada velocidade de predição. No entanto, produzem funções

[4] *SHAP* (*Shapley Additive Explanations*) é um método de XAI fundamentado na teoria dos jogos cooperativos que atribui a cada variável de entrada sua contribuição marginal na predição do modelo, de forma globalmente consistente (NAGAHISARCHOGHAEI et al., 2023).

[5] Modelos de *ensemble* combinam as predições de múltiplos modelos individuais para produzir uma resposta final mais robusta e com melhor generalização do que qualquer modelo isolado.

de resposta descontinuamente constantes por partes, as chamadas respostas *"steppy"*, que comprometem a *smoothness* e dificultam interpretações baseadas em continuidade (BREIMAN, 2001; SAVES, 2025). Redes Neurais Artificiais (ANNs) oferecem alta capacidade de aproximação de funções não-lineares complexas, com acurácia potencialmente superior em espaços de alta dimensionalidade, ao custo de maior exigência de dados para treinamento, ausência de estimativa de incerteza nativa e baixa interpretabilidade intrínseca (GOODFELLOW; BENGIO; COURVILLE, 2016; SAVES, 2025; MOLNAR, 2025). Dado que nenhuma arquitetura domina as demais em todos os atributos, a comparação sistemática entre múltiplos metamodelos é prática recomendada na literatura (SAVES et al., 2024; MONTEVECHI et al., 2023).

A segunda frente trata da eficiência do processo de treinamento por meio do planejamento do espaço amostral, endereçada pelo *Design of Experiments* (DoE). O princípio fundamental é que a qualidade da cobertura do espaço paramétrico afeta diretamente a capacidade de generalização do metamodelo: amostras mal distribuídas elevam a incerteza de predição em regiões não observadas, independentemente da arquitetura utilizada (FORRESTER; SÓBESTER; KEANE, 2008; VIANA, 2016). Com o DoE busca-se, portanto, distribuir as avaliações do simulador de forma representativa com o menor número de execuções possível. No contexto específico de SED, um desafio adicional do DoE é o caráter estocástico do simulador: para um mesmo ponto amostral, execuções distintas do simulador retornam respostas diferentes em função de sementes de aleatoriedade internas ao modelo. Isso exige a realização de múltiplas replicações por ponto para que a média das saídas seja estatisticamente representativa, o que amplifica o custo computacional do processo de geração de dados (MONTEVECHI et al., 2023). A literatura aponta, em função deste desafio, para um dilema de *design* referente à priorização da replicação ou da exploração, uma em detrimento da outra. A replicação envolve treinar o modelo em configurações repetidas para separar o sinal real do ruído nas respostas; enquanto a exploração se refere à busca de uma maior cobertura do espaço de configurações de entrada distintas possíveis. O uso excessivo de ambos em conjunto leva a um espaço amostral denso, contrariando os objetivos do processo de Metamodelagem (BINOIS; GRAMACY; LUDKOVSKI, 2019).

Tomadas as decisões de arquitetura referentes às duas frentes abordadas, entende-se que o modelo pode ser treinado e com isso inicia-se a próxima etapa do *pipeline* citado. Subsequente ao treinamento, a validação é responsável por quantificar a fidelidade do metamodelo ao simulador de referência por meio de métricas como o coeficiente de determinação $R^2$, o erro médio absoluto (MAE) e a raiz do erro quadrático médio (RMSE) (CHAI; DRAXLER, 2014; MONTEVECHI et al., 2023). Essa etapa é essencial para garantir que o metamodelo generalize adequadamente para pontos fora do

conjunto de treinamento antes de ser utilizado em exploração ou otimização.

Na literatura de SED, Montevechi et al. (2023) demonstram a aplicação prática desse *pipeline* em um problema real de alocação de recursos em uma fábrica do setor têxtil modelada via SED. O trabalho emprega LHS para a geração do plano amostral inicial, adota 15 replicações por ponto amostral para lidar com a estocasticidade do simulador, e utiliza um modelo de *ensemble* baseado em *Gradient Boosted Trees* (GBT) com *Bagging* como metamodelo. A validação é conduzida por meio do $R^2$ calculado sobre amostras *out-of-bag* (OOB), e os resultados demonstram que o método proposto supera o *benchmark* baseado em *Kriging* (EGO) em qualidade da solução, mantendo custo computacional equivalente. O estudo reforça que características típicas de SED, como alta estocasticidade, interdependência de processos e possível descontinuidade da função objetivo, favorecem o uso de metamodelos com alta capacidade de aprendizado, como os métodos de *ensemble*. Montevechi et al. (2023) não aplicam camada de explicabilidade sobre o metamodelo elaborado.

A decisão conjunta de arquitetura e DoE deve ser orientada pelos cinco atributos e seus *trade-offs* específicos ao contexto do problema. No âmbito deste trabalho, onde a camada de explicabilidade é requisito funcional, os atributos de *interpretability* e *fast prediction* assumem peso adicional: a velocidade de predição viabiliza a aplicação de métodos de XAI baseados em amostragem em escala global, enquanto a interpretabilidade, seja intrínseca ao modelo ou recuperada via técnicas *post-hoc*, permite transformar o metamodelo em instrumento de suporte à decisão auditável (NAGAHISARCHOGHAEI et al., 2023; SAVES, 2025; MOLNAR, 2025).

### 2.1.4 Explicabilidade

O tema de Explicabilidade conectado ao aprendizado de máquina tem sido uma esfera de pesquisa com tendência de crescimento expressivo desde 2018. O comportamento observado nas publicações dentro desse escopo indica o posicionamento da questão da Explicabilidade como objetivo no topo das prioridades em estudos recentes da área do aprendizado de máquina (SAVES, 2025). Olhando para a ligação entre Explicabilidade e Metamodelagem, não tinham sido desenvolvidos muitos trabalhos associando os dois temas até 2018 e, desde então, vêm ganhando espaço no cenário de pesquisa internacional, apontando para uma tendência de crescimento do foco em lidar com a transparência de metamodelos (SAVES, 2025).

A motivação central do XAI reside no problema da opacidade de modelos de aprendizado de máquina. Ribeiro et al. (2016) formularam essa questão de forma seminal ao argumentar que a adoção de modelos preditivos em ambientes de alto impacto depende não apenas de sua acurácia, mas da capacidade de compreender e justificar suas predições. Modelos complexos, como redes

neurais e métodos de *ensemble*, aprendem representações internas que não se traduzem diretamente em explicações interpretáveis por humanos, configurando o que a literatura denomina de modelos *caixa-preta* (*black box*) (RIBEIRO; SINGH; GUESTRIN, 2016; NAGAHISARCHOGHAEI et al., 2023).

No contexto deste trabalho, é importante distinguir dois tipos de opacidade que coexistem na análise de Sistemas Discretos. O simulador de referência, como o ProModel, é opaco no sentido de que executa sequencialmente a lógica interna do sistema modelado sem atribuir valor à participação de cada parâmetro de entrada no resultado final: é possível observar entradas e saídas, mas não a contribuição marginal de cada variável ao resultado (SAVES, 2025). Embora simuladores como esse possam gravar configurações paramétricas de entrada ótimas e informações relevantes sobre a estrutura física do sistema, geralmente não possuem capacidade nativa de explicar a influência global e local[6] de certos parâmetros nos resultados. Já o metamodelo, por sua vez, herda e por vezes agrava essa opacidade: sendo um modelo de aprendizado de máquina treinado sobre os dados do simulador, ele aproxima o comportamento deste de forma compacta, mas sua estrutura interna tampouco revela diretamente como as variáveis de entrada influenciam as saídas (SAVES, 2025; BODRIA et al., 2021). É sobre essa segunda camada de opacidade que o XAI atua. A aplicação de métodos de Explicabilidade sobre o Metamodelo permite a compreensão das influências das variáveis de entrada nas saídas, atacando também o problema de opacidade do simulador por meio do seu modelo substituto.

A literatura organiza os métodos de XAI em duas grandes taxonomias ortogonais (NAGAHISARCHOGHAEI et al., 2023; PATIL; PATIL, 2023). A primeira distingue métodos *ante-hoc* de métodos *post-hoc*. Métodos *ante-hoc*, também chamados de intrinsecamente interpretáveis, são modelos cuja estrutura é por si só legível (interpretável), como regressões lineares e árvores de decisão: a explicação é imediata, mas frequentemente à custa de acurácia em problemas de alta complexidade (MOLNAR, 2025). Métodos *post-hoc*, por sua vez, são aplicados sobre modelos já treinados para gerar explicações a posteriori, independentemente da arquitetura subjacente, e são os mais relevantes no contexto de metamodelagem por permitirem combinar modelos de alta acurácia com interpretabilidade recuperada (NAGAHISARCHOGHAEI et al., 2023; PATIL; PATIL, 2023). Cabe notar, no entanto, que métodos *post-hoc* introduzem uma camada adicional de aproximação: a explicação gerada nunca é perfeitamente fiel ao comportamento interno do modelo original, existindo sempre uma variação entre o que o método explica e o que o modelo realmente computa (BODRIA et al., 2021; MOLNAR, 2025). A segunda taxonomia distingue métodos globais de métodos locais:

[6]Global considera a influência esperada para uma configuração complementar aleatória, enquanto a Local considera uma influência caracterizada pela vizinhança das configurações complementares possíveis.

métodos globais descrevem o comportamento médio do modelo sobre todo o espaço paramétrico, enquanto métodos locais explicam predições individuais considerando a vizinhança de valores no espaço paramétrico, definindo um comportamento de resposta em função dessas características adjacentes (NAGAHISARCHOGHAEI et al., 2023). Métodos também podem ser classificados entre *model-agnostic* e *model-specific*, representando, no primeiro caso, métodos que podem ser aplicados a qualquer arquitetura de modelo e, no segundo caso, aqueles que só podem ser aplicados sob determinadas circunstâncias.

Entre os métodos *post-hoc* consolidados e relevantes para aplicações em dados tabulares de regressão, como é o caso da metamodelagem de SED, destacam-se três técnicas principais. O SHAP[4] é o método *post-hoc model-agnostic* com maior suporte empírico em dados tabulares, apresentando consistência e fidelidade superiores a alternativas como o LIME[7] em estudos de *benchmarking* (BODRIA et al., 2021). O PDP (*Partial Dependence Plot*) representa a relação média entre uma variável de entrada e a saída do modelo, marginalizando o efeito das demais variáveis, oferecendo uma visão global do comportamento do sistema (NAGAHISARCHOGHAEI et al., 2023). O ICE (*Individual Conditional Expectations*) complementa o PDP ao exibir como essa relação varia para cada observação individual, revelando heterogeneidades que a média do PDP poderia ocultar (NAGAHISARCHOGHAEI et al., 2023). Utilizados em conjunto, SHAP, PDP e ICE oferecem uma visão complementar do comportamento do metamodelo: o SHAP atribui importâncias globais e locais por variável, enquanto PDP e ICE revelam a forma funcional dessas relações ao longo do espaço paramétrico.

Um desafio relevante do estado da arte em XAI, apontado tanto por Nagahisarchoghaei et al. (2023) quanto por Bodria et al. (2021), é a ausência de métricas padronizadas para avaliar a qualidade das explicações geradas. Diferente da acurácia preditiva de um metamodelo, mensurável por métricas como $R^2$ e RMSE, a qualidade de uma explicação é multidimensional: envolve fidelidade ao comportamento do modelo, estabilidade para entradas similares e consistência entre métodos distintos. Essa lacuna é particularmente relevante para o desenvolvimento de um *framework* de metamodelagem explicável, pois os testes de explicabilidade previstos na validação do projeto demandam critérios de avaliação que a literatura ainda não consolidou de forma universal.

## 2.2 Estado da Arte

Na seção anterior (2.1) foi levantada uma serie de temas e conceitos relevantes e como eles se encaixam no contexto de metamodelagem e explicabilidade. Com esse repertório de plano de

[7] *LIME* (*Local Interpretable Model-Agnostic Explanations*) é um método de XAI proposto por Ribeiro et al. (RIBEIRO; SINGH; GUESTRIN, 2016) que gera explicações locais aproximando o comportamento do modelo por um modelo linear simples ao redor de cada predição individual.

fundo, esta seção aborda o Estado da Arte, tratando da integração dessas multiplas frentes e esforços de articulação destas para a produção de metamodelos explicáveis. O entendimento desse panorama atual é útil para delimitar a contribuição que o projeto desenvolvido e explicado ao longo desta monografia tem como proposta situá-lo no contexto da literatura. A seção está organiza em duas frentes, a primeira abordando as sinergias encontradas na articulação dos fundamentos citados, e a segunda focando em projetos similares identificados.

### 2.2.1 Sinergia entre Metamodelagem e XAI

A integração entre Metamodelagem e Inteligência Artificial Explicável constitui o eixo central de contribuição deste trabalho. Embora as duas frentes tenham se desenvolvido de forma relativamente independente na literatura, sua combinação apresenta uma sinergia direta: o metamodelo resolve o problema do custo computacional ao substituir o simulador por um modelo de inferência rápida, enquanto o XAI resolve o problema da opacidade ao tornar as relações entre parâmetros de entrada e saídas do metamodelo interpretáveis e auditáveis (SAVES, 2025).

Saves (2025) aponta que, até 2018, a presença de trabalhos associando explicitamente Metamodelagem e Explicabilidade era limitada na literatura. A partir desse período, o tema passou a ganhar espaço progressivo no cenário de pesquisa internacional, com comportamento descrito pelo autor como em crescimento e tendência identificada como promissora. Esse crescimento reflete uma demanda prática em ambientes de engenharia e consultoria, nos quais não basta que o metamodelo produza predições acuradas, é necessário que os resposáveis por tomarem as decisões compreendam quais parâmetros determinam o comportamento do sistema e em qual nível de impacto, para que as recomendações derivadas do modelo sejam aceitas e implementadas (RIBEIRO; SINGH; GUESTRIN, 2016; SAVES, 2025).

A sinergia entre as duas frentes, no entanto, não é trivial, no qual a escolha da arquitetura do metamodelo afeta diretamente a qualidade e a confiabilidade das explicações geradas pela camada de XAI. Metamodelos com funções de resposta descontínuas, como *Random Forests*, dificultam a interpretação de tendências globais via PDP e ICE, pois as curvas de dependência parcial herdam a natureza escalonada do modelo subjacente (BREIMAN, 2001; SAVES, 2025). Já Metamodelos baseados em Processos Gaussianos produzem resultados que facilitam a interpretação, mas escalam mal com o número de amostras (SAVES et al., 2024). As redes neurais oferecem alta capacidade de aproximação, mas sua opacidade intrínseca exige métodos de XAI mais custosos computacionalmente, como o *KernelSHAP*, em contraste com o *TreeSHAP*, que pode ser calculado analiticamente para modelos baseados em árvores (SAVES, 2025; NAGAHISARCHOGHAEI et al., 2023). Essa

interdependência reforça a necessidade de escolher simultaneamente a arquitetura do Metamodelo e o método de XAI.

Um segundo desafio da integração reside na ausência de métricas padronizadas para avaliar a qualidade das explicações geradas sobre metamodelos. Nagahisarchoghaei et al. (2023) e Bodria et al. (2021) apontam que, diferentemente da acurácia preditiva (mensurável por $R^2$, RMSE e MAE (CHAI; DRAXLER, 2014)), a qualidade de uma explicação é dependente de múltiplos fatores como a fidelidade ao comportamento do modelo, estabilidade nos resultados para entradas similares e consistência entre métodos distintos. No contexto de SED, essa diferença é agravada pela estocasticidade dos simuladores, onde as saídas são geradas a partir das médias de replicações, e as explicações geradas sobre essas médias podem não contemplar a variabilidade inerente ao processo simulado (MONTEVECHI et al., 2023; BINOIS; GRAMACY; LUDKOVSKI, 2019).

### 2.2.2 Projetos Similares

A literatura apresenta trabalhos relacionados à metamodelagem e explicabilidade que, em conjunto, permitem situar o presente projeto no panorama da pesquisa atual. Os trabalhos identificados podem ser organizados em três frentes como feito a seguir.

A primeira frente reúne *toolboxes* e *frameworks* Python de metamodelagem de propósito geral. Bouhlel et al. (2019) introduziram o SMT (*Surrogate Modeling Toolbox*), pacote de código aberto projetado para facilitar a comparação sistemática entre arquiteturas de metamodelo em problemas de engenharia. Saves et al. (2024) publicaram o SMT 2.0, evolução que amplia funcionalidades e integra métodos de amostragem como LHS e sequências quasi-aleatórias às ferramentas disponíveis. Esses trabalhos são relevantes na medida em que demonstram a maturidade do ecossistema Python para metamodelagem, estabelecendo padrões de arquitetura modular e interfaces bem documentadas úteis para orientar a concepção de um *framework* como o proposto nesta monografia. Contudo, esta frente não trata de implementações aplicadas a um problema ou simulador específico: essas ferramentas são instrumentos auxiliares de propósito geral, que não contemplam integração com simuladores de eventos discretos e tampouco incorporam camada de explicabilidade.

A segunda frente reúne trabalhos que integram explicitamente metamodelagem e XAI, mas em contextos distintos do SED industrial. Saves et al. (2025) propõem um *workflow* conceitual (proposta metodológica, sem aplicação funcional) para abordar a Metamodelagem integrada ao XAI, mas o faz no contexto de sistemas multi-agentes. Simulações baseadas em agentes modelam o comportamento coletivo de entidades autônomas e suas interações com aplicações em epidemiologia e dinâmica social. SED, por outro lado, modela ambientes produtivos e logísticos que evoluem em

resposta a eventos pontuais bem definidos. Entretanto, ambas as formas de simulação possuem estocasticidade inerente a seus processos, permitindo analogias metodológicas no que tange o *design* de experimentos (DoE). Ainda assim, representam paradigmas distintos em termos de estrutura, domínio de aplicação e relevância industrial. No domínio de simulações baseadas em equações físicas, como análise de elementos finitos (FEA) e dinâmica dos fluidos computacional (CFD), Zhang et al. (2023) aplicam metamodelos interpretados via SHAP para otimização de compressores axiais e Palar et al. (2024) exploram o espaço de projeto com metamodelos explicáveis em problemas multiobjetivo. Em comum, esses trabalhos, entre outros, demonstram a viabilidade técnica da integração entre metamodelagem e XAI e operam em paradigmas de simulação baseados em equações físicas com foco em engenharia. Dessa forma, estes acabam não abordando o contexto produtivo e logístico característico de aplicações industriais de SED.

A terceira frente concentra os trabalhos mais diretamente relacionados ao presente projeto, por operarem especificamente no contexto de Simulação de Eventos Discretos. Montevechi et al. (2023) propõem um *framework* de otimização via simulação que combina DoE com LHS, ensemble de metamodelos baseados em *Gradient Boosted Trees* e estratégia adaptativa de seleção de pontos amostrais (*infill*), validado em um problema real de alocação de recursos em manufatura têxtil. O estudo demonstra ganhos substanciais de eficiência computacional frente à simulação exaustiva e representa a referência metodológica mais próxima do presente trabalho no que tange à aplicação de metamodelagem em SED industrial. No entanto, o *framework* proposto por Montevechi et al. (2023) não incorpora camada de explicabilidade sobre os metamodelos treinados, limitando-se à função de otimização, sem quantificar a contribuição dos parâmetro de entrada nos indicadores de desempenho.

## 2.3 Conclusões Parciais

A revisão bibliográfica consolidou os quatro campos que sustentam o *framework* desenvolvido neste projeto: Simulação a Eventos Discretos, Projeto de Experimentos (DoE), Metamodelagem e Explicabilidade. A Simulação a Eventos Discretos, embora seja uma metodologia consolidada, impõe duas limitações significativas na sua implementação, o custo computacional e opacidade dos resultados, sobre as quais a Metamodelagem e o XAI atuam de forma complementar. A literatura demonstra que essa sinergia é tecnicamente viável e apresenta tendência de crescimento. No entanto, sua aplicação ao contexto específico de SED na industria ainda é incipiente.

A revisão da pesquisa na área revelou três grupos distintos de projetos similares à proposta de trabalho abordada nesta monografia. O primeiro compreende *toolboxes* Python de metamodelagem de propósito geral, que estabelecem padrões de arquitetura úteis, mas não se propõem à integração

com simuladores específicos nem à incorporação de XAI. O segundo reúne trabalhos que integram metamodelagem e explicabilidade, porém os abordam em simulações baseadas em agentes ou em equações físicas e, estão limitados a aplicações que não abordam SED e nem relevância industrial. O terceiro grupo é o mais próximo ao presente projeto, com aplicações de metamodelagem em SED industrial, mas que ainda não avançaram na incorporação de explicabilidade.

O diferencial do presente trabalho reside, portanto, na união dos conceitos abordados durante a revisão bibliográfica no contexto específico de SED industrial, utilizando o ProModel como simulador de referência, e integrando-o a um *framework* composto das etapas de DoE, Metamodelagem e XAI de forma estruturada e reprodutível para Sistemas a Eventos Discretos genéricos, oportunidade que a literatura ainda não explorou extensivamente. Com a base teórica e o estado da arte consolidados, a próxima seção é dedicada à detalhar e desenvolver o plano de projeto que permitirá elaborar o *framework* proposto nesta monografia.

# 3 PLANO DE PROJETO

Após a consolidação do referencial teórico e a análise do estado da arte apresentados na seção anterior, esta seção dedica-se a detalhar o Plano de Projeto para a construção do *framework* proposto. O objetivo aqui é transpor os procedimentos conceituais identificados para um escopo prático e estruturado de engenharia. Para isso, o capítulo é subdividido de modo a apresentar, primeiramente, uma visão geral da arquitetura da solução abordada na Seção 3.1. Em seguida, são rigorosamente definidos os requisitos funcionais e não-funcionais que guiarão o desenvolvimento e garantirão a validade e usabilidade da ferramenta. Por fim, é detalhada a Metodologia, que atua como o *pipeline* do trabalho, descrevendo o passo a passo sequencial para a execução do projeto, desde a amostragem de dados no simulador até a validação da camada de explicabilidade.

## 3.1 Resumo da Abordagem

O presente trabalho propõe um *framework* que integra duas frentes complementares:

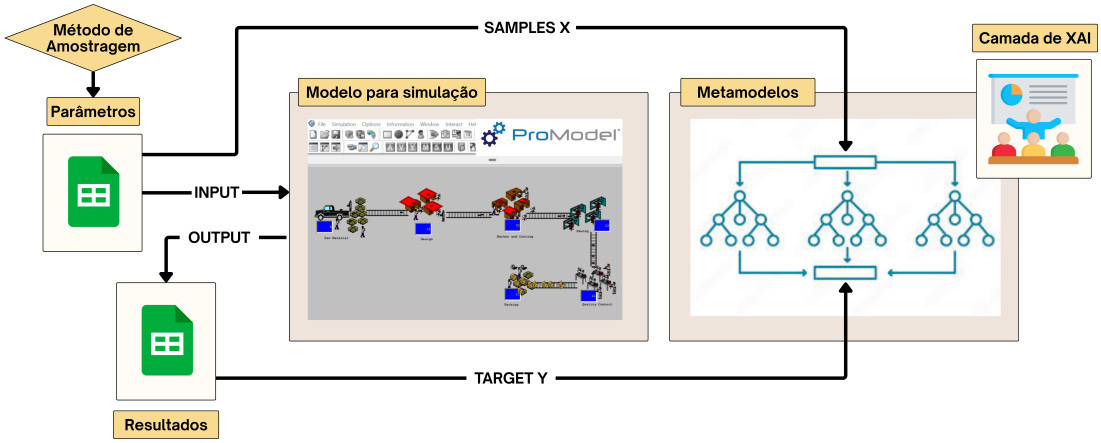
1. **Metamodelagem:** O ProModel atua como simulador de um processo ou sistema produtivo real, gerando conjuntos de dados de relação entrada-saída conforme o espaço amostral definido por meio de um Projeto de Experimentos (DoE). Esses dados são utilizados para treinar modelos de aprendizado de máquina que aproximam o comportamento do simulador com menor custo computacional por inferência, viabilizando a análise de cenários em tempos significativamente menores do que os requeridos pelo simulador original.

2. **Explicabilidade (XAI):** Uma camada de Inteligência Artificial Explicável é aplicada sobre o metamodelo treinado. Utilizam-se técnicas como SHAP *(Shapley Additive Explanations)*, PDP *(Partial Dependence Plots)* e ICE *(Individual Conditional Expectations)* além de explorar outras ferramentas de explicabilidade relevantes na literatura (NAGAHISARCHOGHAEI et al., 2023). Com as técnicas definidas, a contribuição de cada parâmetro operacional nos indicadores de desempenho de interesse é quantificada, tornando o processo decisório auditável e matematicamente justificado.

O *framework* é estruturado como uma cadeia de processamento modular e documentada em Python, aplicável a sistemas SED genéricos. A validação contempla modelos sintéticos de complexidade crescente e um projeto real com dados anonimizados cedidos por empresa de consultoria especializada em modelagem. A Figura 1 é um esquema simplificado dos componentes do projeto

e da forma como se conectam considerando o fluxo de dados entre o Modelo para simulação e os Metamodelos a serem desenvolvidos. Entende-se que o *framework* desenvolvido trata justamente da aplicação prática dessas conexões de forma a consolidar um processo integrado e replicável para outros modelos que possam ser simulados.

Figura 1: Diagrama simplificado das conexões e fluxo do sistema.




**Fonte:** Elaborado pelos autores, adaptado de (MUCHTAR, 2025)

## 3.2 Requisitos

Os requisitos do *framework* proposto foram definidos a partir das limitações identificadas na literatura e das necessidades práticas do contexto de aplicação. Eles descrevem o que o sistema deve ser capaz de fazer e quais restrições deve satisfazer, sendo organizados em duas categorias: requisitos funcionais, que especificam as capacidades esperadas do *framework*, e requisitos não-funcionais, que estabelecem as qualidades e restrições que devem ser atendidas independentemente das funcionalidades implementadas.

### 3.2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as capacidades que o *framework* deve possuir para cumprir seus objetivos. Cada requisito é formulado de forma independente de solução técnica e associado a um critério de verificação mensurável.

1. **Geração de dados via DoE:** O *framework* deve ser capaz de gerar planos amostrais parametrizados e executar a coleta estruturada de dados de entrada e saída do ProModel, cobrindo o espaço paramétrico de forma estatisticamente representativa.

   **Verificação:** O plano amostral gerado deve satisfazer critérios de uniformidade na represen-

tatividade do espaço amostral, e os dados coletados devem estar livres de valores ausentes ou inconsistências estruturais. A replicação de pontos amostrais deve ser capaz de lidar com a estocasticidade do modelo original, prevendo consistentemente uma resposta média esperada similar para diferentes rodadas de treinamento de metamodelos sobre processos que envolvam um comportamento estocástico. Também devem ser considerados os efeitos destas frentes no custo computacional e metas de fidelidade entre o metamodelo e o modelo original.

2. **Treinamento e validação de metamodelos:** O *framework* deve treinar ao menos dois algoritmos de aprendizado de máquina distintos sobre os dados gerados, produzindo modelos capazes de inferir os indicadores de desempenho de saída a partir de combinações de parâmetros de entrada não observadas durante o treinamento. Para cada modelo substituto, deve calcular e reportar métricas de fidelidade, permitindo a seleção fundamentada do metamodelo mais adequado ao problema.

   **Verificação:** Cada metamodelo treinado deve reportar ao menos as métricas de $R^2$, *RMSE* e *MAE*, calculadas no conjunto de validação sobre pontos não utilizados no treinamento e comparadas a *benchmarks* da área.

3. **Aplicação de camada de explicabilidade:** O *framework* deve aplicar ao menos três métodos de XAI *post-hoc* (SHAP, PDP e ICE) sobre o metamodelo selecionado, gerando visualizações que quantifiquem a contribuição marginal de cada variável de entrada nos indicadores de desempenho de saída.

   **Verificação:** As análises SHAP global e local, os gráficos PDP e as curvas ICE devem ser gerados sem erros para qualquer metamodelo suportado pelo *framework*. Dada a dificuldade já comentada na definição de métricas quantitativas para avaliar explicabilidade, o *benchmarking* deve ser feito em contato com especialistas da industria e professores com atução na área, focando em uma validação a partir de projetos reais.

4. **Comparação de latência:** O *framework* deve medir e reportar o tempo de inferência do metamodelo em comparação com o tempo de execução equivalente no simulador ProModel, para o mesmo conjunto de cenários avaliados.

   **Verificação:** O relatório de *benchmarking* deve apresentar a razão entre os tempos de inferência do metamodelo e do simulador, demonstrando redução significativa de latência.

5. **Aplicabilidade genérica a modelos SED:** O *framework* deve ser aplicável a modelos SED distintos sem alteração de sua estrutura modular, requerendo apenas a configuração dos parâ-

metros de entrada, saída e do plano amostral específico de cada problema.

**Verificação:** O *framework* deve ser executado com sucesso em ao menos dois modelos SED distintos (um sintético e um caso real) sem modificações no código-fonte dos módulos principais.

### 3.2.2 Requisitos Não-Funcionais

Os requisitos não-funcionais descrevem restrições e qualidades que o *framework* deve satisfazer, independentemente das funcionalidades específicas que implementa.

1. **Modularidade:** O *framework* deve ser estruturado em módulos funcionalmente independentes (geração de dados, pré-processamento, treinamento, validação e explicabilidade) com interfaces claramente definidas entre eles, de modo que cada módulo possa ser substituído ou estendido sem impacto nos demais.
   **Verificação:** Cada módulo deve ser executável de forma isolada a partir de entradas e saídas padronizadas, documentadas na especificação de arquitetura.

2. **Reprodutibilidade:** Dada a mesma semente aleatória e o mesmo conjunto de dados de entrada, o *framework* deve produzir resultados idênticos em execuções distintas, garantindo a replicabilidade dos experimentos.
   **Verificação:** Duas execuções independentes com os mesmos parâmetros devem produzir métricas de validação e visualizações de XAI idênticas.

3. **Documentação:** O código deve ser documentado de forma suficiente para que um usuário com conhecimento básico em Python e aprendizado de máquina seja capaz de aplicar o *framework* a um novo modelo SED sem suporte direto dos autores.
   **Verificação:** Presença de *docstrings* em todas as funções públicas e de um guia de uso cobrindo o fluxo completo de execução.

4. **Desempenho computacional:** O tempo total de treinamento e validação do metamodelo (excluindo o tempo de execução das simulações no ProModel) deve ser significativamente inferior ao tempo equivalente de execução direta no simulador para o mesmo conjunto de cenários avaliados, de modo que o ganho de latência justifique a etapa de treinamento.
   **Verificação:** Medição e comparação dos tempos de execução dos módulos de treinamento e validação frente ao tempo de simulação equivalente no ProModel, em ambiente de referência documentado.

5. **Rastreabilidade:** Todos os artefatos gerados pelo *framework* (conjuntos de dados, modelos treinados, métricas e visualizações) devem ser versionados e associados aos parâmetros de configuração que os produziram, permitindo auditoria posterior dos resultados.
   **Verificação:** Cada execução do *framework* deve gerar um arquivo de registro (*log*) contendo os parâmetros de configuração utilizados e os caminhos dos artefatos produzidos.

## 3.3 Metodologia

Esta seção descreve a abordagem técnica e processual adotada para o desenvolvimento do *framework*, detalhando as decisões metodológicas que orientam cada etapa da cadeia de processamento proposta. A construção de uma ferramenta robusta de metamodelagem explicável exige a orquestração dessas diversas etapas interdependentes, portanto, para que o processo seja lógico, modular e replicável, a metodologia proposta foi estruturada em cinco fases sequenciais, as quais correspondem ao passo a passo da operação do *framework* e cujo planejamento será detalhado nas subseções abaixo de acordo com a organização a seguir:

1. **Projeto de Experimentos (DoE):** O planejamento de como o espaço paramétrico será amostrado para garantir a máxima representatividade estatística com o menor custo computacional;

2. **Simulador e Aplicação no Espaço Amostral:** A configuração do simulador de referência (ProModel) e a execução em lote dos cenários planejados para a extração do conjunto de dados relacionando entradas e saídas;

3. **Metamodelos e Treinamento:** A definição das arquiteturas de *Machine Learning* (como *Random Forests* ou Redes Neurais) e o processo de aprendizado sobre os dados amostrados;

4. **Testes e Plano de Validação:** A avaliação do desempenho preditivo dos metamodelos, comparando sua latência e fidelidade (utilizando métricas como $R^2$, RMSE e MAE) em relação ao simulador original;

5. **Camada de Explicabilidade:** A aplicação técnica de métodos de XAI sobre o metamodelo selecionado para tornar transparente a influência de cada variável de entrada nos resultados do sistema.

Entende-se, no entanto, que o processo de estruturação dessas fases sequenciais não é linear, a elaboração prática de um *framework* que lide com essas diversas etapas interdependentes envolve um processo cíclico de prototipagem e teste para refinar e concluir quais as melhores abordagem em cada uma das frentes da metodologia, como será explicado nas seções seguintes.

### 3.3.1 Ciclo de Desenvolvimento

A metodologia para o desenvolvimento do *framework* de metamodelagem explicável foi abordada de forma iterativa. A elaboração da ferramenta parte de hipóteses baseadas na teoria, como a descrita por Forrester (2008) e estudo de casos similares como o de Montevechi (2023), mas, à medida que avança, é ajustada e conduzida pelos resultados de iterações anteriores.

Esse ciclo de ajustes e análises permite entender o funcionamento do *framework*, suas limitações e suas vantagens. Entretanto, para alcançar um bom resultado, é importante que esse processo ciclíco seja sistemático, garantindo que as iterações culminem em avanços e não numa exploração aleatória de possibilidades. Para tal, 3 pontos da abordagem devem ser definidos. São eles:

- **Os valores iniciais de cada parâmetro**, baseados na teoria e em precedentes da literatura;
- **Os gatilhos de revisão**, isto é, quais resultados observados levariam a ajustar qual parâmetro, em qual direção e por qual razão;
- **Os limites do espaço de busca**, ou seja, o que está fora da região relevante para o projeto e, portanto, não será testado independentemente dos resultados, por razões de custo computacional ou incoerência com os objetivos.

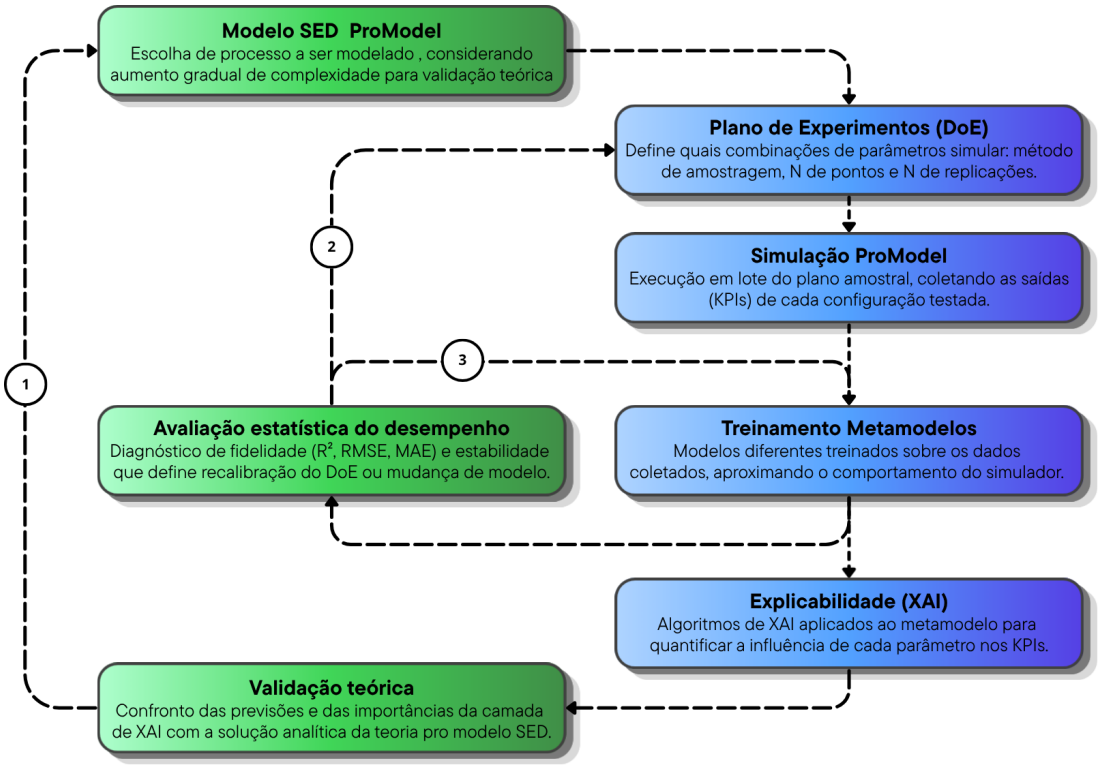
A Figura 2 ilustra o procedimento iterativo: o ciclo se inicia na escolha do Modelo SED a ser simulado no ProModel e segue, em sequência, para o Plano de Experimentos (DoE), a Simulação no ProModel e o Treinamento de Metamodelos, onde existe uma primeira parada para avaliar estatisticamente o desempenho do processo até o momento.

É nesse ponto, na Avaliação Estatística, que o ciclo deixa de ser estritamente sequencial. Em vez de seguir adiante para a Explicabilidade, o resultado da avaliação decide qual das frentes anteriores precisa ser revisitada, e essa decisão dá origem a fluxos de retorno destacados na Figura 2.

O fluxo 3, de menor escopo, retorna da Avaliação Estatística diretamente ao Treinamento de Metamodelos, sem alterar o plano amostral já coletado. É o fluxo acionado sempre que o problema identificado pode ser resolvido ajustando o próprio modelo, como ao comparar *Random Forest*, Processo Gaussiano e Rede Neural sobre o mesmo conjunto de dados, ou ao reajustar hiperparâmetros de um modelo específico. Pela correção do modelo ser o ajuste mais barato entre os três fluxos, é também o primeiro a ser tentado em cada iteração, e cada passagem por ele é registrada, de modo que a comparação sistemática entre arquiteturas surja do histórico de iterações.

Somente quando o fluxo 3 se esgota, isto é, quando trocar de modelo ou reajustar hiperparâmetros deixa de produzir ganhos relevantes nas métricas de fidelidade, é que se justifica acionar o

Figura 2: Diagrama do ciclo de desenvolvimento do *framework*.




**Fonte:** Elaborado pelos autores.

fluxo 2. Esse fluxo também parte da Avaliação Estatística, mas retorna ao Plano de Experimentos, reconfigurando o método de amostragem, o número de pontos ou o número de replicações dentro dos limites definidos previamente. Uma vez reconfigurado o DoE, o ciclo volta a percorrer Simulação, Treinamento e Avaliação, podendo alternar entre os fluxos 2 e 3 quantas vezes forem necessárias até que ambos os subciclos estejam satisfeitos.

O fluxo 1, de maior escopo, só é acionado depois que os fluxos 2 e 3 se esgotam em conjunto, isto é, quando nem o ajuste de modelo nem o ajuste do plano amostral produzem mais melhorias dentro do nível de complexidade do sistema de eventos discretos corrente. Nesse ponto, o ciclo segue adiante para a Explicabilidade e, em seguida, para a Validação Teórica, onde as previsões do metamodelo e as importâncias extraídas pela camada de XAI são confrontadas com a solução esperada para o sistema simulado. Esse confronto assume forma distinta conforme o caso: para os modelos sintéticos desenvolvidos no ProModel, a validação é feita por comparação direta com a solução analítica conhecida para o tipo de sistema escolhido; para o caso real, sem solução analítica, ela é conduzida por meio da literatura de metamodelagem e da coerência dos resultados com o que já está consolidado sobre o processo modelado.

O resultado da Validação Teórica fecha o ciclo retornando ao Modelo SED, e autoriza uma de

duas mudanças. Quando um problema sistemático é identificado, como uma importância de variável que contraria de forma persistente a relação esperada, o retorno é estrutural: revisam-se premissas já estabelecidas nos fluxos 2 e 3 antes de prosseguir. Quando, ao contrário, a concordância entre o *framework* e a teoria é satisfatória, o retorno é de progressão: avança-se para o próximo nível de complexidade do sistema simulado, reiniciando o ciclo sob uma configuração inicial já informada pelo aprendido na etapa anterior.

Essa hierarquia entre os três fluxos garante que o processo iterativo não seja uma exploração aleatória do espaço de configurações possíveis: o ciclo sempre tenta primeiro o ajuste de menor escopo e custo, e só recorre a um ajuste de maior escopo depois que o anterior se esgota e é devidamente documentado.

### 3.3.2 Modelo de Sistema de Eventos Discretos

Esta frente trata da escolha dos sistemas a serem modelados no ProModel, decisão que precede e condiciona todas as demais etapas da metodologia, uma vez que define os parâmetros de entrada disponíveis para o DoE, os indicadores de desempenho que servem de saída para o treinamento dos metamodelos e a existência ou não de uma solução analítica de referência para a Validação Teórica descrita na Seção 3.3.8. A estratégia adotada é a progressão gradual de complexidade: parte-se de um modelo SED simples com solução analítica conhecida, avança-se por cenários de complexidade crescente e, por fim, aplica-se o *framework* a um caso industrial real. Essa progressão serve a um propósito central: garantir que o *framework* seja validado em condições controladas antes de ser submetido a um problema sem referência exata de comparação, conforme o ciclo de desenvolvimento descrito na Seção 3.3.1.

Os três cenários sintéticos desenvolvidos no ProModel são descritos a seguir.

**Cenário 1 - Linha Sequencial Simples:** O sistema mais básico consiste em uma linha de produção com fluxo único e sequencial: os produtos entram pelo ponto de entrada e percorrem três processos em série até atingir a saída (Figura 3). Cada processo possui um único recurso de atendimento e tempos de processo modelados por distribuições com parâmetros controláveis. A escolha por esse sistema como ponto de partida se justifica pela existência de solução analítica completa para os indicadores de desempenho típicos de SED, como o tempo médio de espera em fila, a utilização dos recursos e o *throughput*, deriváveis diretamente da teoria de filas (SCHRIBER; BRUNNER, 2003). Essa referência exata permite separar com clareza se uma divergência observada no metamodelo é causada por limitação do *framework* ou por falha na modelagem do sistema no ProModel, conferindo a esta etapa a função de calibração e verificação inicial de todo o processo.

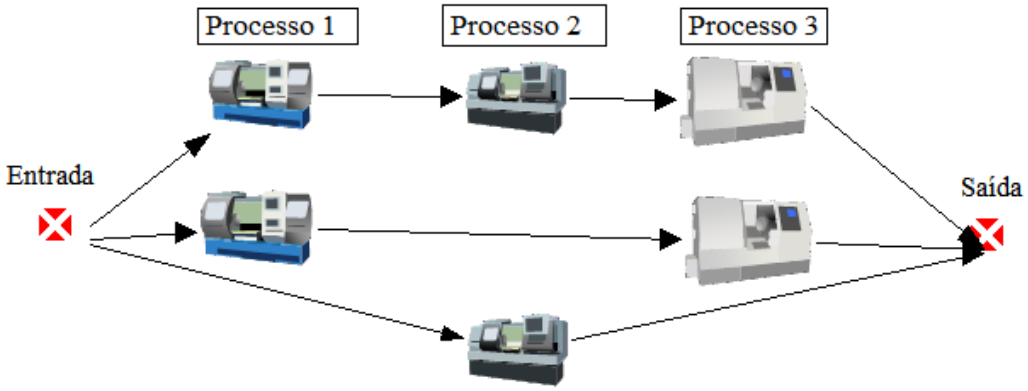
Figura 3: Cenário 1 - linha sequencial com três processos em série.




**Fonte:** Elaborado pelos autores.

**Cenário 2 - Linha com Roteamento Probabilístico:** O segundo cenário introduz uma bifurcação logo após o ponto de entrada: o produto é encaminhado aleatoriamente para um de três caminhos possíveis, com três, dois ou apenas um processo de atendimento antes de atingir a saída (Figura 4). Em relação ao Cenário 1, este acrescenta dois elementos novos: a aleatoriedade do roteamento, que determina qual caminho cada entidade percorre, e a variabilidade no número de etapas cumpridas por produto, gerando diferenças no tempo total de permanência no sistema em função do caminho sorteado. O metamodelo precisa, portanto, capturar não apenas os efeitos isolados dos parâmetros de cada processo, mas também as interações entre a taxa de chegada, as probabilidades de roteamento e o balanceamento de carga entre os caminhos. A solução analítica existe para casos particulares desse tipo de sistema (GROSS et al., 2008), permitindo ainda alguma verificação direta, porém de forma mais limitada do que no Cenário 1.

Figura 4: Cenário 2 - roteamento probabilístico para três caminhos de comprimento distinto.

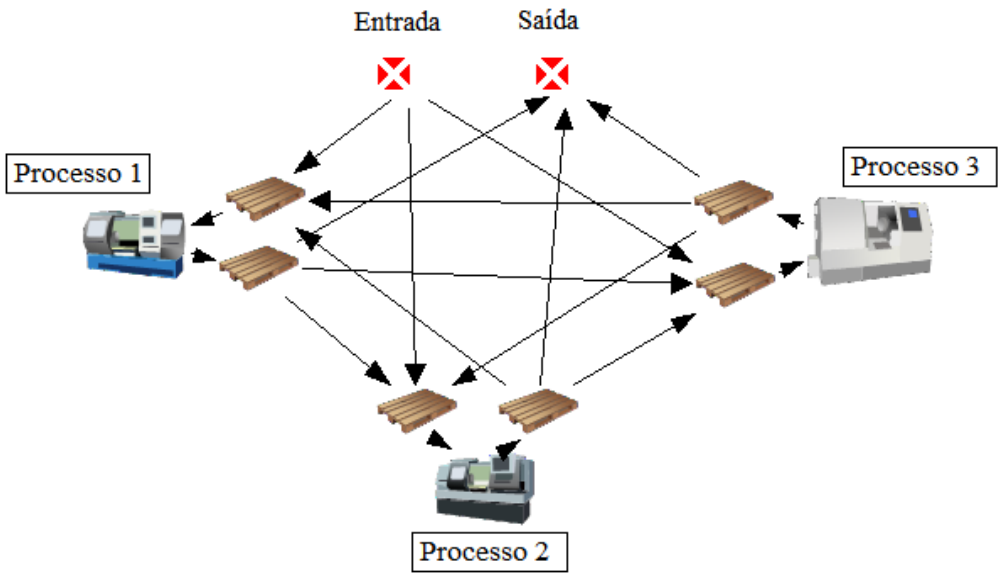



**Fonte:** Elaborado pelos autores.

**Cenário 3 - Sistema Multi-Produto com Sequências Distintas:** O terceiro cenário é o mais complexo entre os sintéticos: diferentes tipos de produto entram pelo mesmo ponto de entrada e seguem sequências de processos distintas, determinadas pelo tipo, até atingirem a saída (Figura 5). A complexidade aumenta substancialmente em relação ao Cenário 2 porque diferentes tipos de produto competem pelos mesmos recursos, gerando filas conjuntas nos processos compartilhados. O com-

portamento emergente desse sistema, como a formação de gargalos, a variação dos tempos de espera em função do mix de produtos e a interação entre os diferentes fluxos, não possui solução analítica fechada e não pode ser previsto com precisão por análise manual (GROSS et al., 2008). É nesse cenário que o metamodelo deixa de ser apenas uma ferramenta de verificação cruzada e passa a ser uma forma viável de explorar o espaço paramétrico de forma sistemática, justificando sua inclusão como último nível sintético da progressão.

Figura 5: Cenário 3 - sistema multi-produto com sequências de processos distintas por tipo de produto.




**Fonte:** Elaborado pelos autores.

A Tabela 1 resume as principais características dos três cenários sintéticos e os objetivos específicos de cada um no processo de validação do *framework*.

A progressão entre os cenários segue a lógica do fluxo 1 descrito na Seção 3.3.1: o avanço ao próximo nível só ocorre após a concordância satisfatória entre o *framework* e a solução esperada no nível corrente. O Cenário 1 serve de âncora de calibração; o Cenário 2 verifica se a fidelidade se mantém quando se introduz aleatoriedade estrutural; e o Cenário 3 testa o comportamento do *framework* em um sistema cujos resultados só podem ser observados empiricamente via simulação.

Após a validação nos três cenários sintéticos, o *framework* é aplicado a um modelo industrial real cedido por empresa parceira de consultoria especializada em modelagem e simulação de sistemas logísticos e produtivos. Esse modelo, disponibilizado em formato anonimizado, representa um sistema de complexidade superior à dos cenários sintéticos, com parâmetros, distribuições e interdependências provenientes de uma operação real. A ausência de solução analítica para esse caso exige

Tabela 1: Comparação dos cenários sintéticos desenvolvidos no ProModel.

| Cenário | Estrutura | Complexidade | Solução analítica | Objetivo no ciclo |
|---|---|---|---|---|
| **1** | Linha sequencial com 3 processos em série | Baixa | Disponível para todos os indicadores | Calibração e validação direta do *framework* |
| **2** | Roteamento probabilístico para 3 caminhos de comprimentos distintos | Média | Parcialmente disponível | Teste da capacidade de capturar aleatoriedade no roteamento e interações entre caminhos |
| **3** | Multi-produto com sequências de processos distintas por tipo | Alta | Não disponível | Validação em ambiente sem referência analítica exata |

que a validação seja conduzida por meio da coerência dos resultados com a literatura de metamodelagem aplicada e com o julgamento de especialistas, conforme descrito na Seção 3.3.8.

### 3.3.3 Projeto de Experimentos (DoE)

Nesta frente, deve ser definida: a combinação de parâmetros de entrada do SED modelado; a estratégia e método de amostragem do espaço paramétrico resultante da escolha anterior; o número de pontos amostrais a serem escolhidos e o número de replicações por ponto.

A primeira decisão é a estratégia de amostragem, que pode ser fixa ou incremental. Embora a estratégia incremental, como descrito na seção 2.1.2, seja mais eficiente, optou-se por explorar apenas a abordagem fixa ao passo que a incremental exigiria o acomplamento dos módulos de DoE e de treinamento, contrariando o requisito de modularidade. A abordagem fixa possibilita o funcionamento isolado das peças do *framework* e garante que possam ser utilizadas junto a diferentes blocos de treinamento em outros projetos, tornando essa peça mais replicável em novos cenários de SED. A exploração da alternativa incremental pode ser vista como uma evolução natural deste projeto para casos mais específicos.

A segunda decisão é o método de preenchimento do espaço paramétrico dentro da estratégia fixa escolhida. A Tabela 2 aborda os 3 métodos, além de como e quando devem ser usados, baseando-se nas informações discutidas na seção 2.1.2. O número de pontos amostrais por método varia principalmente em função do comportamento observado na avaliação estatística do desempenho em casos *underfitting* ou *overfitting*.

A terceira decisão é a quantidade de replicações por ponto amostral. O número é pesquisado dentro da faixa de 3 a 30: o piso de três garante o cálculo de uma média e desvio amostral, e o teto de

Tabela 2: Comparação das técnicas de amostragem consideradas no DoE.

| Método | Papel no estudo | Pontos positivos | Pontos negativos |
|---|---|---|---|
| **Monte Carlo** | ***Baseline*.** Referência de desempenho contra a qual LHS e Sobol são comparados e avaliados. | Implementação trivial e sem suposições sobre o espaço. | Menor desempenho esperado pois tende a formar aglomerações e vazios no espaço paramétrico. |
| **LHS** | **Método principal.** Adotado por padrão para dimensão média-baixa do espaço paramétrico de entrada. | Cobertura uniforme de cada variável, robusto e de fácil implementação. | Ganho sobre Sobol diminui em alta dimensão. |
| **Sobol** | **Alternativa.** Reservado para alta dimensionalidade ($\gtrsim 10$ variáveis). | Convergência superior em alta dimensão. | Benefício marginal em baixa dimensão. |

trinta é o dobro do precedente mais próximo na literatura (MONTEVECHI et al., 2023), definido para garantir o orçamento computacional necessário para cobrir o espaço paramétrico, condição alinhada ao requisito de Desempenho Computacional. Dentro dessa faixa, o número efetivamente adotado é determinado por um critério de precisão relativa baseado no intervalo de confiança (IC) das saídas do simulador (LAW, 2007): partindo do piso mínimo, replicações são adicionadas incrementalmente e, a cada incremento, avalia-se a razão entre a o tamanho da incerteza no IC (IC de 95% por convenção), ou seja, quanto a média pode efetivamente variar para mais ou para menos, e a média estimada da métrica de saída. O processo continua até que essa razão, satisfaça um limiar pré-definido, ou até que o teto de trinta replicações seja atingido, validando se o limiar desejado é alcançável dentro das restrições computacionais do projeto. Esse limiar corresponde à tolerância de precisão relativa, e é tratado como um parâmetro configurável do módulo de DoE do *framework*, podendo ser ajustado ao longo do desenvolvimento ou em evoluções subsequentes do projeto conforme as características estocásticas de cada sistema simulado.

A revisão desta frente ocorre por meio do fluxo 2 descrito na Figura 2, acionado quando a Avaliação Estatística do Desempenho, indica que o ajuste isolado de modelo não é mais suficiente para melhorar a fidelidade do *framework*.

### 3.3.4 Simulador e Aplicação no Espaço Amostral

Esta frente corresponde à execução em lote do plano amostral definido na Seção 3.3.3 sobre o sistema configurado no ProModel, conforme a Seção 3.3.2. A entrada desta etapa é o conjunto de combinações paramétricas a serem simuladas e a saída é o conjunto de dados de entrada e saída coletado para cada ponto e cada replicação, organizado em formato tabular para consumo direto pela

etapa de Metamodelos e Treinamento descrita na Seção 3.3.5.

A automação desta etapa depende da integração nativa do ProModel com planilhas do Excel, que permite a parametrização sistemática das entradas e a exportação estruturada das saídas sem intervenção manual entre execuções. Diferentemente das demais frentes da metodologia, esta etapa não possui parâmetros próprios de ajuste, já que toda a sua configuração é herdada do plano amostral definido anteriormente, e por isso não é, isoladamente, ponto de origem de nenhum dos três fluxos do ciclo de desenvolvimento.

### 3.3.5 Metamodelos e Treinamento

Nesta frente, devem ser definidas as arquiteturas de aprendizado de máquina utilizadas como metamodelos, o pré-processamento necessário para cada uma e o critério de seleção do modelo a ser levado às etapas seguintes. Os modelos iniciais escolhidos são três: *Random Forest*, Processo Gaussiano e Rede Neural Artificial, selecionados de modo a cobrir posições distintas no espaço de *trade-offs* entre os atributos relevantes ao *framework* como um todo, discutidos na Seção 2.1.3. A Tabela 3 resume essa comparação e explicita por que nenhuma das três famílias é descartada de antemão.

Tabela 3: Comparação qualitativa das famílias de metamodelo consideradas.

| Família | Acurácia | Suavidade | Vel. inferência | Incerteza | Explicabilidade |
|---|---|---|---|---|---|
| **Random Forest** | Alta | Baixa | Alta | Estimada indiretamente | Exata e eficiente |
| **Processo Gaussiano** | Alta* | Alta | Moderada | Nativa ao modelo | Aproximada |
| **Rede Neural** | Alta | Moderada | Moderada | Estimada indiretamente | Aproximada |

*Condicionada a volumes moderados de dados, pelo custo cúbico de treinamento discutido na Seção 2.1.3.

**Fonte:** Elaborado pelos autores com base em (BREIMAN, 2001), (FORRESTER; SÓBESTER; KEANE, 2008) e (GOODFELLOW; BENGIO; COURVILLE, 2016).

A leitura cruzada da Tabela 3 evidencia as razões práticas de cada escolha no contexto do *framework*. A velocidade de inferência refere-se ao tempo de resposta do modelo já treinado, distinta do custo de treinamento. A incerteza indica se o modelo produz uma estimativa de confiança nativa à sua formulação, conforme discutido na Seção 2.1.3. A explicabilidade indica se a contribuição de cada variável pode ser calculada de forma exata, como no *Random Forest* via TreeSHAP, ou apenas aproximada por amostragem, como no Processo Gaussiano e na Rede Neural via KernelSHAP.

Do ponto de vista do comportamento durante o treinamento, o *Random Forest* tende a ser resistente ao sobreajuste, mas com capacidade limitada de representar funções suaves (BREIMAN, 2001). O Processo Gaussiano ajusta-se bem com poucos dados, mas pode falhar em relações de alta complexidade (FORRESTER; SÓBESTER; KEANE, 2008). A Rede Neural tem maior capacidade de representar funções complexas, mas exige volumes de dados maiores para generalizar de forma confiável, tornando-a mais suscetível ao sobreajuste nos volumes típicos de um DoE em SED (GOODFELLOW; BENGIO; COURVILLE, 2016). O diagnóstico dessas situações e as ações correspondentes são tratados na Seção 3.3.6.

O ajuste de hiperparâmetros é realizado por validação cruzada dentro de cada família, explorando parâmetros como número de árvores e profundidade no *Random Forest*, função de *kernel* no Processo Gaussiano, e número de camadas e neurônios na Rede Neural, sempre dentro dos limites de regime que justificaram a inclusão de cada família. Arquiteturas adicionais, como *Gradient Boosted Trees*, são mantidas como alternativas documentadas para aprofundamentos após os testes com as três famílias indicadas.

A revisão desta frente ocorre por meio do fluxo 3 descrito na Figura 2, de menor escopo entre os três fluxos e o primeiro a ser percorrido em cada iteração. Os principais gatilhos de revisão estão resumidos na Tabela 4.

Tabela 4: Gatilhos de revisão de Metamodelos e Treinamento.

| **Observação** | **Interpretação** | **Ajuste** |
|---|---|---|
| Diferença relevante entre métricas de treino e validação | Sobreajuste (*overfitting*) | Simplificar arquitetura ou aumentar regularização |
| Métricas baixas tanto em treino quanto em validação | Subajuste (*underfitting*) | Aumentar capacidade do modelo |
| Instabilidade entre sementes aleatórias distintas | Treinamento não convergiu de forma confiável | Revisar hiperparâmetros antes de prosseguir |

**Fonte:** Elaborado pelos autores com base em (GOODFELLOW; BENGIO; COURVILLE, 2016).

### 3.3.6 Testes e Plano de Avaliação Estatística

Esta frente avalia o desempenho preditivo dos metamodelos treinados na Seção 3.3.5, correspondendo ao bloco de Avaliação Estatística do Desempenho no ciclo de desenvolvimento descrito na Seção 3.3.1. É nesta etapa que se decide se uma nova iteração deve ser conduzida pelo fluxo 3, ajustando o modelo, pelo fluxo 2, ajustando o plano amostral, ou se ambos os subciclos já atingiram um patamar de estabilização.

A avaliação numérica combina três métricas, resumidas na Tabela 5. Cada uma é sensível a um padrão de erro distinto: um metamodelo com MAE baixo e RMSE alto, por exemplo, acerta bem na maior parte do espaço mas erra de forma acentuada em pontos específicos.

Tabela 5: Métricas de fidelidade preditiva adotadas na avaliação.

| **Métrica** | **O que mede** | **Diagnóstico característico** |
|---|---|---|
| $R^2$ | Proporção da variância da saída explicada pelo metamodelo | Qualidade geral do ajuste; pouco sensível à magnitude absoluta do erro |
| **RMSE** | Raiz do erro quadrático médio | Penaliza erros grandes; sensível a pontos atípicos |
| **MAE** | Erro médio absoluto | Erro típico esperado; robusto a pontos atípicos |

**Fonte:** Elaborado pelos autores com base em (CHAI; DRAXLER, 2014).

Além das métricas numéricas, são avaliados de forma mais qualitativa o erro em função do volume de dados utilizados e o tempo de treinamento em função do volume de dados. As métricas numéricas são o principal instrumento de diagnóstico de sobreajuste (*overfitting*) e subajuste (*underfitting*). Já as avaliações qualitativas descritas permitem identificar o ponto em que o custo computacional de uma família cresce de forma acentuada, sinalizando o limite prático de uso dentro do orçamento disponível.

A avaliação é conduzida por validação cruzada durante o ajuste de hiperparâmetros e por um conjunto de teste *hold-out* independente para a seleção final do metamodelo, garantindo uma medida não enviesada pelo conjunto de treino.

A progressão esperada ao longo das iterações é de ganhos decrescentes nas métricas de fidelidade até um patamar de estabilização. É esse patamar que sinaliza o esgotamento de um fluxo e sinaliza que seria apropriado o acionamento do fluxo de escopo imediatamente superior, conforme descrito na Seção 3.3.1. Os gatilhos que, de forma simplificada, determinam qual tipo de ajuste faz sentido aplicar e em qual frente estão resumidos na Tabela 6.

Tabela 6: Gatilhos de revisão da Avaliação Estatística do Desempenho.

| Observação | Interpretação | Ajuste no modelo | Ajuste no DoE |
|---|---|---|---|
| Curva de validação diverge da curva de treinamento | Sobreajuste (não generaliza bem para pontos fora do conjunto de treino) | Simplificar arquitetura ou aumentar regularização | Aumentar o número de pontos amostrais e/ou replicações. |
| Curvas de treino e validação convergem para erro elevado | Subajuste (não captura bem tendência nos dados) | Aumentar capacidade do modelo | Revisar a estratégia amostral |
| MAE baixo combinado a RMSE alto | Erro concentrado em região específica do espaço | Revisar hiperparâmetros | Revisar estratégia amostral na região problemática |
| Variância elevada entre partições de validação cruzada | Instabilidade durante o treinamento | Revisar hiperparâmetros | Investigar número de replicações |

**Fonte:** Elaborado pelos autores com base em (GOODFELLOW; BENGIO; COURVILLE, 2016), (CHAI; DRAXLER, 2014) e (BINOIS; GRAMACY; LUDKOVSKI, 2019).

### 3.3.7 Camada de Explicabilidade

Esta frente aplica métodos de XAI sobre o metamodelo selecionado na Seção 3.3.5, com o objetivo de quantificar a contribuição de cada parâmetro de entrada nos indicadores de desempenho de saída. As técnicas adotadas são complementares entre si e estão resumidas na Tabela 7.

Tabela 7: Técnicas de explicabilidade aplicadas e a pergunta que cada uma responde.

| Técnica | Pergunta respondida | Observação |
|---|---|---|
| **SHAP** | Qual a contribuição marginal de cada variável para uma predição específica | Implementação depende da arquitetura: TreeSHAP para Random Forest, KernelSHAP para os demais |
| **PDP** | Como a saída média varia em função de uma variável, mantendo as demais fixas | Mais informativo para metamodelos com função de resposta suave |
| **ICE** | A relação entre uma variável e a saída é homogênea ou varia entre subgrupos | Revela interações que o PDP, por ser uma média, pode ocultar |

**Fonte:** Elaborado pelos autores com base em (MOLNAR, 2025).

A implementação do SHAP é definida em função da arquitetura do metamodelo selecionado na Seção 3.3.5: o TreeSHAP é utilizado para o Random Forest, por permitir o cálculo exato e eficiente das contribuições aproveitando a estrutura de árvore; o KernelSHAP, aplicável a qualquer modelo, é utilizado para o Processo Gaussiano e a Rede Neural, ao custo de um cálculo por aproximação amostral.

Os resultados desta frente alimentam diretamente a Validação Teórica descrita na Seção 3.3.8.

Para os sistemas sintéticos, espera-se que as importâncias extraídas sigam o comportamento previsto pela teoria e solução analítica associadas a problemas similares aos das formulações sintéticas de SED abordadas. Divergências sistemáticas entre o resultado obtido e o esperado constituem o principal insumo analítico do fluxo 1 do ciclo de desenvolvimento. Resultados adequados indicam que o próximo passo é aumentar a complexidade do modelo SED para testar a efetividade do *framework* em um novo cenário.

### 3.3.8 Plano de Validação Teórica

Esta frente confronta as previsões do metamodelo e as importâncias extraídas pela camada de Explicabilidade, descrita na Seção 3.3.7, com a solução esperada para o sistema modelado. Corresponde ao bloco de Validação Teórica no ciclo de desenvolvimento e é o ponto de origem do fluxo 1, acionado apenas após o esgotar os fluxos 2 e 3 dentro do nível de complexidade corrente.

A metodologia se altera conforme o caso testado, como indicado na Tabela 8.

Tabela 8: Metodologia de Validação Teórica por tipo de caso.

| **Caso** | **Referência de validação** | **Procedimento** |
|---|---|---|
| **Sintético** | Solução analítica da teoria de filas | Comparação direta entre previsões do metamodelo e valores exatos, e entre importâncias do SHAP e as derivadas analíticas esperadas |
| **Real** | Literatura de metamodelagem e especialistas do processo | Avaliação de coerência dos resultados com casos semelhantes documentados, complementada por consulta a especialistas quando viável |

Nos sistemas sintéticos com solução analítica, a comparação é feita sem execuções adicionais do ProModel, tanto para as previsões do metamodelo quanto para as importâncias da camada de Explicabilidade, que podem ser confrontadas com as derivadas analíticas dos indicadores em relação a cada parâmetro. Nos níveis sem solução fechada e no caso real, a validação passa a depender da coerência com a literatura e, quando viável, da avaliação de especialistas. A confiança estabelecida nos níveis sintéticos serve de base para sustentar os resultados nos níveis subsequentes.

Os resultados desta frente configuram dois tipos de retorno ao bloco do Modelo SED:

- mudança de progressão, quando a concordância com a solução esperada é satisfatória, autorizando o avanço ao próximo nível de complexidade;
- mudança estrutural, quando um problema sistemático é identificado, exigindo revisão de premissas estabelecidas nas frentes anteriores antes de prosseguir.

# 4 IMPLEMENTAÇÃO

Esta seção descreve o trabalho de implementação da infraestrutura de código e dos modelos e rotinas de simulação conforme o plano apresentado nas seções anteriores. É dada ênfase à abordagem geral em detrimento do preciosismo técnico nas explicações que seguem, no entanto, maior profundidade no âmbito técnico de desenvolvimento do projeto pode ser obtida explorando os Apêndices: B e C. Cada subseção registra a aplicação dos requisitos de projeto nas diferentes etapas de sua construção e consolidação como um *framework* prático.

## 4.1 Arquitetura do projeto

O requisito não-funcional de modularidade foi tratado como restrição de projeto. Na prática, isso levou cada etapa da cadeia a ser construída de modo a ser executável de forma isolada, a partir de entradas e saídas tabulares explícitas, sem que nenhuma precise conhecer o funcionamento interno das demais.

Essa abordagem guiou a implementação do acoplamento entre as etapas. Sempre que duas etapas precisaram trocar informação, essa informação é registrada em um formato inspecionável, como uma tabela com colunas nomeadas, ao invés de trafegar por dentro do código, na memória de execução. Isso torna cada fronteira entre etapas um ponto de verificação possível e facilita o processo de integração do *framework* a outros ambientes e problemas caso seja desejado.

A arquitetura de código foi elaborada com uma divisão em módulos responsáveis por subblocos internos aos processos de amostragem, simulação, metamodelagem e explicabilidade. A quebra foi elaborada da seguinte forma:

- **Bloco de amostragem.** Traduz a declaração do espaço de parâmetros em uma lista de execuções a simular:
  - **Módulo de projeto de experimentos (DoE)**: gera o plano amostral, um conjunto de pontos em que cada ponto é uma combinação distinta de valores dos parâmetros de entrada, de modo que o espaço declarado seja preenchido segundo o método de amostragem escolhido entre os previstos na Seção 3.3.3. Esse conjunto produzido é retornado em formato tabular onde cada amostra é uma linha;
  - **Módulo de replicação**: expande cada ponto do plano em várias execuções independentes, para que a estocasticidade do simulador seja amostrada. Na prática, esta etapa pode ser vista como a duplicação das linhas do formato tabular retornado pelo Módulo de projeto

de experimentos.

- **Bloco de interface com o simulador.** Isola o restante da cadeia do formato exigido pelo ProModel:
  - **Módulo de ponte com o *cockpit***: escreve o plano de execução na planilha de *cockpit* no formato consumido pelo ProModel e a lê de volta após as simulações, restaurando a natureza declarada de cada coluna, ou seja, se contém valores numéricos ou categóricos, inteiros ou flutuantes, além de classificar o estado de cada execução em função dos resultados obtidos, indicando falhas na simulação se ocorrerem em determinada linha.
- **ProModel.** Etapa da cadeia externa ao *framework*, executada no simulador:
  - **Planilha de *cockpit***: arquivo Excel em formato tabular, com uma linha por execução, as colunas de parâmetros de entrada preenchidas e as colunas de indicadores de saída reservadas para os resultados;
  - **Modelos de simulação**: arquivos do ProModel que representam o sistema de eventos discretos, leem a planilha de *cockpit*, executam uma simulação por linha com os parâmetros ali declarados e gravam os indicadores de saída de volta na mesma planilha.
- **Bloco de pré-processamento.** Prepara para o aprendizado os dados lidos pelo bloco de interface:
  - **Módulo de *pipeline* de pré-processamento**: codifica os parâmetros de entrada conforme a natureza declarada de cada um, aplica a cada indicador de saída uma transformação escolhida previamente e separa os conjuntos de treinamento e teste, agrupando por ponto do plano amostral. Produz as matrizes de entrada e os vetores de saída consumidos pelo bloco de metamodelos.
- **Bloco de metamodelos.** Reúne as famílias de modelos substitutos, todas expostas sob uma mesma assinatura de função, requerendo parâmetros comuns e, assim, facilitando a comparação, teste e seleção de modelos:
  - **Módulo de modelos de referência**: preditor da média, regressões lineares e polinomiais, com e sem regularização;
  - **Módulo da família de árvores de decisão**: conjuntos de árvores construídos por agregação e por reforço sequencial;

  - **Módulo da família de processos gaussianos**: variantes homocedásticas e variante com modelagem explícita de ruído de intensidade variável;
  - **Módulo da família de redes neurais**: perceptron multicamadas.

- **Bloco de avaliação estatística.** Mede a fidelidade dos metamodelos sobre o conjunto de teste e devolve as previsões às unidades originais dos indicadores (em construção).
- **Bloco de explicabilidade (XAI).** Aplica as técnicas de atribuição de importância sobre o metamodelo selecionado (em construção).
- **Bloco de validação teórica.** Confronta previsões e importâncias com a solução esperada para o sistema modelado (em construção).

A decisão de elaborar as funções de cada módulo de metamodelo tal que expusessem suas variantes sob uma mesma assinatura foi guiada pelos requisitos de modularidade, com o objetivo de ter um sistema com peças intercambiáveis, de modo que treinar ou ajustar hiperparâmetros de qualquer uma delas não exige conhecer a biblioteca que a implementa. O detalhamento das funções de cada módulo está no Apêndice C.

## 4.2 Geração do plano amostral

Esta etapa traduz a definição do espaço paramétrico em um conjunto de pontos a simular. O espaço é declarado externamente ao código, em arquivo de configuração, e admite cinco naturezas de parâmetro: contínuo, inteiro, categórico nominal, categórico ordenado e binário.

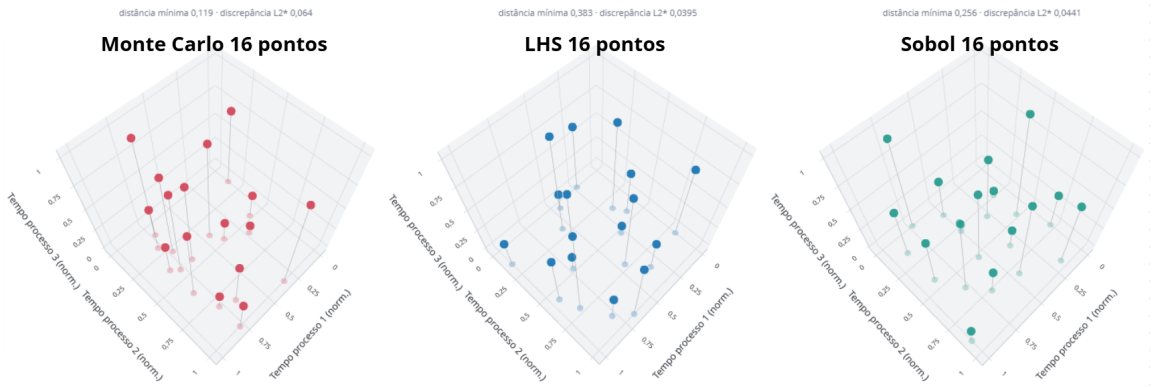
Implementaram-se os métodos de amostragem previstos para o projeto usando a biblioteca SMT para a amostragem estratificada, a biblioteca SciPy para a sequência de baixa discrepância e o gerador de números aleatórios do NumPy para a amostragem aleatória simples, além da implementação de funcionalidades não inclusas e adaptações.

Entre essas adaptações está o caminho de conversão dos pontos gerados para o espaço declarado. Os três métodos preenchem um hipercubo unitário e um único procedimento converte esse preenchimento para os domínios de cada parâmetro. Parâmetros contínuos são reescalados para o intervalo declarado, e parâmetros discretos recebem faixas de largura igual, de forma que todos os seus níveis sejam equiprováveis. Manter a conversão comum aos três métodos é o que permite que eles sejam comparados entre si, já que passam a diferir apenas na forma de preencher o hipercubo.

A Figura 6 ilustra a diferença de preenchimento do hipercubo unitário entre os três métodos, alavancando os resultados de testes da implementação descrita nesta subseção em um exemplo

tridimensional com 16 pontos. A amostragem aleatória simples tende a formar aglomerações e vazios, enquanto o LHS e a sequência de baixa discrepância (Sobol) apresentam uma cobertura mais uniforme do espaço.

Figura 6: Comparação ilustrativa entre os métodos de amostragem no hipercubo unitário.




**Fonte:** Elaborado pelos autores.

### 4.3 Tratamento da estocasticidade por replicação

Como o simulador é estocástico, cada ponto do plano amostral é executado mais de uma vez. A implementação expande o plano em uma lista de execuções na qual cada linha corresponde a uma execução individual. Além dos parâmetros de entrada, repetidos em todas as replicações de um mesmo ponto, cada linha recebe 2 campos de controle: o identificador do ponto de origem e o índice geral da replicação dentro do conjunto completo de dados.

A expansão é feita de modo a preservar a natureza declarada de cada parâmetro, de forma que parâmetros inteiros, categóricos e ordenados cheguem ao bloco de interface com a mesma tipagem com que saíram do bloco de amostragem.

O número de replicações por ponto é um parâmetro de configuração do estudo. O valor deste parâmetro é definido antes de rodar o processo de amostragem, e deve ser ajustado diretamente no arquivo de configuração do cenário sendo testado. Entende-se que cada problema possui um determinado nível de estocasticidade, e que o número de replicações pode gerar resultados distintos em diferentes problemas. Assim, a ideia da definição prévia é permitir uma escolha orientada do operador ou analista, que conhece o processo sendo modelado, e incentivar a revisão sistemática do número escolhido segundo o ciclo iterativo descrito na Seção 3.3.1.

## 4.4 Interface com o simulador

A integração com o ProModel é feita por planilha, aproveitando sua capacidade nativa de leitura e escrita nesse formato, conforme previsto na Seção 3.3.4. A planilha é organizada de modo que cada linha corresponda a uma execução, com os parâmetros de entrada à esquerda e os indicadores de saída à direita. As colunas de saída são entregues vazias, e a célula em branco funciona como marcador de que aquela execução ainda não foi realizada. Isso permite que o simulador percorra a planilha sequencialmente e que o progresso seja inspecionável durante uma campanha longa.

Na leitura, cada execução é classificada em um de quatro estados: completa, quando todos os indicadores foram preenchidos com valores numéricos; parcial, quando apenas alguns foram; pendente, quando nenhum foi, que é o estado normal de uma campanha em andamento; e inválida, quando algum indicador foi preenchido com conteúdo não numérico. A classificação não remove linhas. A seleção das execuções aproveitáveis é uma etapa separada, o que mantém as execuções anômalas visíveis e localizáveis na planilha de origem e permite distinguir uma falha de coleta de um resultado legítimo do sistema.

## 4.5 ProModel

A Seção 3.3.2 apresentou a proposta conceitual dos três cenários sintéticos de complexidade crescente que compõem a etapa de Modelo de Sistema de Eventos Discretos do *framework*, delimitando a estrutura, os objetivos e o papel de cada um no ciclo de desenvolvimento. Esta seção descreve a implementação efetivamente realizada dessa proposta no ProModel: os três modelos foram construídos, juntamente com os códigos responsáveis por sua lógica interna e os *cockpits* que concentram as entradas e saídas de cada simulação. Sobre o *software*, descreve-se o funcionamento e ferramentas do ProModel no Apêndice D, de modo a situar tecnicamente as decisões de implementação apresentadas na sequência.

### 4.5.1 Implementação do Cenário 1 - Linha Sequencial Simples

O Cenário 1, apresentado conceitualmente na Seção 3.3.2, foi implementado como uma linha de produção com fluxo único: as entidades entram pelo local de Entrada, percorrem sequencialmente os processos 1, 2 e 3 (cada um executado por uma única máquina) e deixam o sistema pelo local de Saída (Figura 3).

Todos os parâmetros que caracterizam o comportamento do sistema são controlados diretamente pelo *cockpit* (Figura 7): a capacidade de cada máquina, isto é, o número de entidades que

ela processa simultaneamente; a quantidade de entidades por chegada, o número de ocorrências de chegada e a frequência entre elas; e o tempo de processo de cada máquina. Como as entidades são de um único tipo e a sequência de processamento é fixa (Entrada $\rightarrow$ Processo 1 $\rightarrow$ Processo 2 $\rightarrow$ Processo 3 $\rightarrow$ Saída), a capacidade de cada máquina em relação à quantidade e à frequência das chegadas determina diretamente a formação de filas: sempre que o número de entidades aguardando um processo supera a capacidade instantânea da máquina correspondente, forma-se um gargalo que se propaga ao restante da linha.

O acompanhamento desse comportamento é feito por meio dos atributos e variáveis descritos no Apêndice D, que alimentam diretamente as células de saída do *cockpit*:

- `Entradas` e `Prontas`: contabilizam, respectivamente, o número de entidades que entraram no sistema e o número de entidades que completaram integralmente o processamento e saíram. A comparação entre os dois valores verifica se todas as entidades que entram no sistema efetivamente o deixam ao final da simulação;
- `Tempo_ate_P1`, `Tempo_ate_P2` e `Tempo_ate_P3`: tempo médio decorrido entre a entrada da entidade no sistema e sua chegada a cada processo;
- `Tempo_P1`, `Tempo_P2` e `Tempo_P3`: tempo médio observado em cada processo. Como o tempo de processo nominal já é conhecido (é um parâmetro de entrada), um valor observado superior ao nominal evidencia a ocorrência de um gargalo que impede a entidade de seguir imediatamente a lógica de processamento;
- `Feito_P1`, `Feito_P2` e `Feito_P3`: quantidade de entidades que passaram por cada processo;
- `Tempo_total`: tempo médio total de permanência das entidades no sistema, entre sua entrada e sua saída;
- `Tempo_simul`: tempo total de simulação, medido entre a entrada da primeira entidade e a saída da última.

Por ser o cenário de menor complexidade, com sequência fixa e solução analítica conhecida (Seção 3.3.2), esse conjunto de indicadores é suficiente para caracterizar integralmente o comportamento do sistema, servindo de base direta de comparação com os valores obtidos pela teoria de filas na etapa de Validação Teórica (Seção 3.3.8).

### 4.5.2 Implementação do Cenário 2 - Roteamento Probabilístico

O Cenário 2, apresentado conceitualmente na Seção 3.3.2, mantém os mesmos mecanismos de controle de capacidade, chegadas e tempos de processo do Cenário 1, mas introduz uma bifurcação logo após a Entrada: cada entidade passa a seguir um entre três caminhos possíveis até atingir a Saída (Figura 4).

Os três caminhos diferem no número e na identidade dos processos percorridos:

- **Caminho 1 (superior):** Processo 1 $\rightarrow$ Processo 2 $\rightarrow$ Processo 3 $\rightarrow$ Saída;
- **Caminho 2 (intermediário):** Processo 1 $\rightarrow$ Processo 3 $\rightarrow$ Saída;
- **Caminho 3 (inferior):** Processo 2 $\rightarrow$ Saída.

Embora todas as entidades sejam do mesmo tipo, a escolha do caminho seguido por cada uma delas é definida por um sorteio executado pela subrotina de roteamento (Apêndice D) no momento em que a entidade entra no sistema, atribuindo-lhe um caminho conforme uma distribuição de probabilidade discreta. Esse mecanismo é o mesmo que caracteriza o cenário como de **roteamento probabilístico** na Seção 3.3.2, introduzindo uma aleatoriedade estrutural distinta daquela já presente nos tempos de processo e nas chegadas.

Como os três caminhos têm comprimentos e composições distintas, os indicadores descritos para o Cenário 1 deixam de ser suficientes isoladamente: para captar corretamente o comportamento do sistema, todos eles passam a ser calculados de forma segregada por caminho e, dentro de cada caminho, por processo percorrido. Isso permite observar particularidades da operação de cada caminho (como o tempo total de permanência e a ocupação das máquinas compartilhadas) e como o sistema reage à interação entre eles, uma vez que os caminhos competem pelos mesmos processos em pontos distintos de sua sequência.

### 4.5.3 Implementação do Cenário 3 - Sistema Multi-Produto

O Cenário 3, apresentado conceitualmente na Seção 3.3.2, preserva a lógica geral de controle dos cenários anteriores, mas reorganiza a disposição física do modelo e introduz um novo elemento estrutural: cada uma das três máquinas (uma por processo) passa a contar com um *buffer* de entrada e um *buffer* de saída próprios, de capacidade suficientemente grande para não restringir a operação (Figura 5).

Esses *buffers* são necessários porque, neste cenário, os três caminhos possíveis se cruzam nos mesmos processos em posições diferentes da sequência:

- **Caminho 1:** Processo 1 → Processo 2 → Processo 3 → Saída;
- **Caminho 2:** Processo 3 → Processo 2 → Processo 1 → Saída;
- **Caminho 3:** Processo 2 → Processo 3 → Processo 1 → Saída.

Como a mesma máquina ocupa posições distintas em cada caminho, uma entidade que aguarda para ser processada pode bloquear a máquina que a antecede na sequência de outro caminho, propagando o bloqueio (*blocking*) para trás até travar o sistema por completo. Os *buffers* de entrada e saída eliminam esse risco: uma entidade que termina o processamento é sempre encaminhada ao *buffer* de saída, liberando a máquina imediatamente, independentemente de o próximo processo de seu caminho estar disponível; de forma análoga, uma entidade só é processada quando chega sua vez no *buffer* de entrada da máquina correspondente. A escolha do caminho segue o mesmo mecanismo de sorteio probabilístico do Cenário 2 (Seção 4.5.2).

Esse cruzamento entre caminhos e processos é o que torna o Cenário 3 o mais complexo dos três: diferente dos anteriores, nele não é possível estimar manualmente, com precisão, os tempos médios e a ocupação de cada máquina, pois esses valores dependem da interação dinâmica entre os três fluxos de entidades disputando os mesmos recursos em ordens diferentes, por este motivo a ausência de solução analítica fechada já apontada na Seção 3.3.2 e a necessidade de recorrer à simulação para obter esses resultados, o que torna este cenário um teste particularmente relevante para o metamodelo. Os indicadores de saída seguem a mesma estrutura do Cenário 2, segregados por caminho e, dentro de cada caminho, por processo, assim, como as máquinas são compartilhadas entre os três caminhos, o tempo observado em cada processo passa a variar conforme o caminho que a entidade percorre, mesmo tratando-se da mesma máquina física.

### 4.5.4 *Cockpit*, Execução em Lote e Preparação de Dados para o DoE

Além do *cockpit* individual de cada cenário, descrito nas seções anteriores, foi implementada uma segunda camada de planilha responsável por organizar e executar em lote o conjunto de simulações necessário para compor a base de dados de treinamento dos metamodelos (Figura 7).

Nessa planilha, cada linha corresponde a um cenário de simulação, isto é, a uma combinação específica de valores para os parâmetros de entrada do modelo. As colunas à esquerda (região de Entrada) armazenam os valores de cada parâmetro para aquela linha, e as colunas à direita (região de Saída) recebem os indicadores de desempenho exportados ao final da simulação correspondente. Essa estrutura é o que operacionaliza, na prática, a integração entre o Projeto de Experimentos (DoE) e o simulador descrita na Seção 3.3.3: para cada parâmetro de entrada, define-se um intervalo de

Figura 7: Planilha de execução em lote - cada linha corresponde a um cenário simulado.

| | Entrada | | | | | | | | | Saída | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Item | Capacidad | Quantidad | Ocorrênci | Frequênci | Tempo pr | Capacidad | Capacidad | Tempo pr | Tempo pr | Entradas | Prontas | Tempo_at | Tempo_at | Tempo_at | Tempo_P | Tempo_P | Tempo_P | Feito_P1 | Feito_P2 | Feito_P3 | Tempo_to | Tempo_simul |
| Teste 1 | 1 | 1 | 1 | 1 | 5 | 1 | 1 | 6 | 7 | 1 | 1 | 0 | 300 | 660 | 300 | 360 | 420 | 1 | 1 | 1 | 1080 | 1080 |
| Teste 2 | 1 | 1 | 2 | 1 | 5 | 1 | 1 | 6 | 7 | 2 | 2 | 120 | 450 | 840 | 330 | 390 | 420 | 2 | 2 | 2 | 1260 | 1500 |
| Teste 3 | 1 | 1 | 3 | 1 | 5 | 1 | 1 | 6 | 7 | 3 | 3 | 260 | 620 | 1020 | 360 | 400 | 420 | 3 | 3 | 3 | 1440 | 1920 |
| Teste 4 | 1 | 1 | 4 | 1 | 5 | 1 | 1 | 6 | 7 | 4 | 4 | 420 | 795 | 1200 | 375 | 405 | 420 | 4 | 4 | 4 | 1620 | 2340 |
| Teste 5 | 1 | 1 | 5 | 1 | 5 | 1 | 1 | 6 | 7 | 5 | 5 | 588 | 972 | 1380 | 384 | 408 | 420 | 5 | 5 | 5 | 1800 | 2760 |
| Teste 6 | 1 | 1 | 6 | 1 | 5 | 1 | 1 | 6 | 7 | 6 | 6 | 760 | 1150 | 1560 | 390 | 410 | 420 | 6 | 6 | 6 | 1980 | 3180 |
| Teste 7 | 1 | 1 | 7 | 1 | 5 | 1 | 1 | 6 | 7 | 7 | 7 | 934.286 | 1328.57 | 1740 | 394.286 | 411.429 | 420 | 7 | 7 | 7 | 2160 | 3600 |
| Teste 8 | 1 | 1 | 8 | 1 | 5 | 1 | 1 | 6 | 7 | 8 | 8 | 1110 | 1507.5 | 1920 | 397.5 | 412.5 | 420 | 8 | 8 | 8 | 2340 | 4020 |
| Teste 9 | 1 | 1 | 9 | 1 | 5 | 1 | 1 | 6 | 7 | 9 | 9 | 1286.67 | 1686.67 | 2100 | 400 | 413.333 | 420 | 9 | 9 | 9 | 2520 | 4440 |
| Teste 10 | 1 | 1 | 10 | 1 | 5 | 1 | 1 | 6 | 7 | 10 | 10 | 1464 | 1866 | 2280 | 402 | 414 | 420 | 10 | 10 | 10 | 2700 | 4860 |
| Teste 11 | 1 | 1 | 11 | 1 | 5 | 1 | 1 | 6 | 7 | 11 | 11 | 1641.82 | 2045.45 | 2460 | 403.636 | 414.545 | 420 | 11 | 11 | 11 | 2880 | 5280 |
| Teste 12 | 1 | 1 | 12 | 1 | 5 | 1 | 1 | 6 | 7 | 12 | 12 | 1820 | 2225 | 2640 | 405 | 415 | 420 | 12 | 12 | 12 | 3060 | 5700 |
| Teste 13 | 1 | 1 | 13 | 1 | 5 | 1 | 1 | 6 | 7 | 13 | 13 | 1998.46 | 2404.62 | 2820 | 406.154 | 415.385 | 420 | 13 | 13 | 13 | 3240 | 6120 |
| Teste 14 | 1 | 1 | 14 | 1 | 5 | 1 | 1 | 6 | 7 | 14 | 14 | 2177.14 | 2584.29 | 3000 | 407.143 | 415.714 | 420 | 14 | 14 | 14 | 3420 | 6540 |
| Teste 15 | 1 | 1 | 15 | 1 | 5 | 1 | 1 | 6 | 7 | 15 | 15 | 2356 | 2764 | 3180 | 408 | 416 | 420 | 15 | 15 | 15 | 3600 | 6960 |
| Teste 16 | 1 | 1 | 16 | 1 | 5 | 1 | 1 | 6 | 7 | 16 | 16 | 2535 | 2943.75 | 3360 | 408.75 | 416.25 | 420 | 16 | 16 | 16 | 3780 | 7380 |
| Teste 17 | 1 | 1 | 17 | 1 | 5 | 1 | 1 | 6 | 7 | 17 | 17 | 2714.12 | 3123.53 | 3540 | 409.412 | 416.471 | 420 | 17 | 17 | 17 | 3960 | 7800 |
| Teste 18 | 1 | 1 | 18 | 1 | 5 | 1 | 1 | 6 | 7 | 18 | 18 | 2893.33 | 3303.33 | 3720 | 410 | 416.667 | 420 | 18 | 18 | 18 | 4140 | 8220 |
| Teste 19 | 1 | 1 | 19 | 1 | 5 | 1 | 1 | 6 | 7 | 19 | 19 | 3072.63 | 3483.16 | 3900 | 410.526 | 416.842 | 420 | 19 | 19 | 19 | 4320 | 8640 |
| Teste 20 | 1 | 1 | 20 | 1 | 5 | 1 | 1 | 6 | 7 | 20 | 20 | 3252 | 3663 | 4080 | 411 | 417 | 420 | 20 | 20 | 20 | 4500 | 9060 |

**Fonte:** Elaborado pelos autores.

variação (mínimo e máximo), utilizado pelo plano amostral (segundo o método de amostragem definido, conforme a Tabela 2) para gerar o conjunto de combinações a serem simuladas. Esse conjunto é então inserido na planilha, uma combinação por linha, cada uma correspondendo a uma simulação a ser executada.

A execução manual de dezenas ou centenas de simulações, alterando os parâmetros de entrada e coletando as saídas uma a uma, inviabilizaria a escala de amostragem necessária ao treinamento dos metamodelos. Para automatizar esse processo, foi utilizada a ferramenta *Macro* do ProModel, que permite vincular o valor de uma variável a um índice numérico atualizado automaticamente a cada simulação executada. Esse índice é empregado como ponteiro da linha corrente da planilha: a cada simulação, o modelo lê os parâmetros de entrada da linha indicada pelo índice, executa a simulação e exporta os indicadores de saída para essa mesma linha, ao término, o índice é incrementado e a simulação seguinte já é executada sobre a próxima linha, sem necessidade de reconfiguração manual entre execuções. Esse mecanismo é o que viabiliza, na prática, a automação apontada na Seção 3.3.4 para a etapa de Simulador e Aplicação no Espaço Amostral.

Ao final da execução em lote, obtém-se uma única tabela contendo, para cada cenário simulado, o conjunto completo de parâmetros de entrada e os indicadores de saída correspondentes. Essa tabela constitui diretamente o conjunto de dados de treinamento dos metamodelos, no qual a variação sistemática de cada parâmetro de entrada ao longo dos diferentes cenários simulados é o que, posteriormente, permite à camada de Explicabilidade (Seção 3.3.7) quantificar a contribuição de cada parâmetro sobre os indicadores de saída do sistema.

### 4.5.5 Verificação dos Modelos Implementados

Antes de qualquer análise dos resultados numéricos produzidos pelos três cenários, cada modelo foi submetido a uma etapa de verificação, distinta da validação abordada na Seção 3.3.8. Enquanto a validação confronta o comportamento do modelo com uma referência externa a ele (um sistema real ou uma solução analítica), a verificação confirma que o modelo implementado no ProModel corresponde exatamente à lógica especificada para ele (SCHRIBER; BRUNNER, 2003), ou seja, que a construção do modelo está livre de erros de programação independentemente de quão realista seu comportamento resultante venha a ser.

Para os três cenários, a verificação consistiu em confirmar que (i) a sequência de processos percorrida pelas entidades corresponde exatamente à lógica de roteamento definida para cada cenário e, nos Cenários 2 e 3, para cada um dos três caminhos possíveis; e que (ii) os indicadores exportados para o *cockpit* são calculados conforme especificado no Apêndice D, as contagens de entradas, saídas

e peças processadas por processo fecham corretamente, e os tempos médios refletem as médias sobre o total de peças produzidas, com exceção do tempo total de simulação. Essa verificação é o que permite atribuir, com segurança, qualquer divergência observada nas etapas seguintes ao comportamento do sistema modelado, e não a um erro na implementação da lógica do modelo.

## 4.6 Pré-processamento

O pré-processamento é uma etapa crucial de tratamento de dados cujo objetivo é preservar a informação relevante do plano amostral enquanto simultaneamente prepara os dados para o aprendizado de máquina, etapa a ser realizada pelos metamodelos. No geral, modelos de *machine learning* exigem que os dados sejam normalizados entre 1 e 0 para que um viés não seja introduzido pela diferença de escala entre paramêtros. Assim, o processo de normalização, e como ele é realizado, é um ponto crítico de implementação.

A definição de como os dados serão normalizados e pré-processados é uma escolha explícita de quem utiliza o *framework*, e não um comportamento automático. O tratamento adequado depende da natureza de cada parâmetro de entrada e de cada indicador de saída, e essa é uma informação que pertence ao analista que conhece o sistema modelado, não ao código. O módulo oferece as transformações e exige que sejam declaradas, em vez de inferi-las dos dados, assim evitando que o sistema faça assumpções prejuciais e o tornando mais adaptável ao sistema modelado.

Para os parâmetros de entrada, a codificação é derivada dos limites declarados no arquivo de configuração do espaço amostral. Parâmetros contínuos e inteiros são reescalados para o intervalo unitário usando os valores mínimo e máximo declarados, e parâmetros categóricos são codificados a partir da lista de níveis declarada. A vantagem dessa abordagem de normalização pelos limites declarados apriori é que ela restringe o problema: os limites são conhecidos antes de qualquer simulação, fazem parte da definição do espaço de busca e delimitam a região dentro da qual o metamodelo será treinado e utilizado.

Para os indicadores de saída a situação é diferente, porque sua faixa de variação só existe depois de simular. A transformação escolhida para cada indicador é ajustada exclusivamente sobre o conjunto de treinamento para evitar o vazamento de informações, e os conjuntos de treinamento e teste são transformados com os parâmetros obtidos. Estão disponíveis a padronização, a transformação logarítmica seguida de padronização e a ausência de transformação. A escolha é feita por indicador, e a transformação inversa correspondente é fornecida para que as previsões possam ser reportadas nas unidades originais.

A separação entre treinamento e teste tem um requisito de implementação que decorre da

replicação. Como todas as replicações de um mesmo ponto do plano amostral compartilham exatamente os mesmos valores de entrada, a separação precisa ser feita por ponto original, e não por linha. Caso fosse feita por linha, algumas replicações de um ponto ficariam no treinamento e outras no teste, e o metamodelo seria avaliado sobre combinações de parâmetros que já viu durante o treinamento, medindo apenas a capacidade do metamodelo de recuperar informações que já foram vistas, ao invés de inferir o resultado de novos pontos.

É para viabilizar essa separação que o identificador de ponto criado na etapa de replicação é transportado ao longo de toda a cadeia, atravessando a planilha de troca com o simulador e chegando ao pré-processamento. Ele não é um parâmetro do modelo mas é o que permite reconhecer quais linhas pertencem ao mesmo ponto do plano. O módulo utiliza esse identificador para agrupar a separação sempre que os dados estão na granularidade replicada, e essa decisão é tomada pelo próprio código a partir da presença de identificadores repetidos, sem depender de configuração adicional.

## 4.7 Famílias de metamodelos

Foram implementadas quatro famílias de metamodelo. Em cada uma, múltiplas variantes foram disponibilizadas sob a mesma assinatura, para permitir que a comparação entre elas seja feita experimentalmente na etapa de avaliação, em vez de decidida a priori. Junto de cada variante é fornecida uma grade de hiperparâmetros sugerida para o regime de dados deste trabalho, que é o de poucas centenas de pontos amostrais.

A família de **modelos de referência** reúne o preditor da média, que ignora as entradas, e regressões lineares e polinomiais, com e sem regularização. Ela cumpre duas funções: o preditor da média estabelece o piso de comparação, abaixo do qual um metamodelo não justifica seu custo, e as regressões polinomiais correspondem à superfície de resposta clássica do projeto de experimentos. A implementação dos métodos disponíveis nesta família usa majoritariamente a biblioteca *scikit-learn*.

A família de **árvores de decisão** reúne conjuntos construídos por agregação de árvores independentes (*Bagging*) e por reforço sequencial (*Boosting*). As variantes por agregação e por reforço recebem grades de hiperparâmetros distintas, porque o número de árvores é um parâmetro de ajuste apenas no segundo caso (BREIMAN, 2001). A implementação usa principalmente *scikit-learn* e *XGBoost*.

A família de **processos gaussianos** reúne variantes que diferem principalmente na forma como tratam ruído. A implementação usa *scikit-learn*, a biblioteca SMT e a biblioteca hetGPy. Como as duas últimas não seguem a mesma interface de programação do *scikit-learn*, foram escritos adaptadores que uniformizam a assinatura, sem alterar os modelos.

A família de **redes neurais** contém um perceptron multicamadas, implementado com *scikit-learn*. O escopo da profundidade desta família e o número de variantes foi mantido reduzido porque redes neurais requerem volumes de dados superiores ao regime deste trabalho, no qual cada ponto amostral corresponde a uma campanha de simulação com replicações.

Todas as famílias operam sobre os dados já transformados pelo pré-processamento. A conversão das previsões de volta às unidades originais é atribuição da etapa de avaliação, que dispõe dos parâmetros da transformação, o que evita que cada família precise conhecer como os dados foram preparados.

## 4.8 Avaliação estatística

(trabalho em progresso)

## 4.9 Camada de explicabilidade

(trabalho em progresso)

## 4.10 Validação teórica

(trabalho em progresso)

# Apêndice A RELATÓRIO DE ACOMPANHAMENTO DO TCC

## Situação em setembro de 2026

O planejamento metodológico do 1º semestre está concluído. No 2º semestre, o desenvolvimento do *framework* tornou-se a atividade principal e encontra-se parcialmente concluído, conforme registrado na Seção 4.

**Partes já funcionais:** o bloco de amostragem, que gera o plano amostral e o expande em plano de execução replicado; o bloco de interface com o simulador, que escreve e lê a planilha de cockpit; o bloco de pré-processamento, que codifica entradas, transforma indicadores de saída e separa treinamento e teste; e o bloco de metamodelos, com as quatro famílias previstas expostas sob assinatura comum. A cadeia é executável de ponta a ponta, da declaração do espaço de parâmetros ao treinamento de um metamodelo.

**Partes em construção:** os blocos de avaliação estatística, de explicabilidade e de validação teórica, além dos módulos de controle do ciclo de desenvolvimento e de rastreabilidade. A geração de dados via DoE está em andamento e a aplicação ao caso real depende da cessão dos dados industriais pela empresa parceira.

O Diagrama de Gantt (Figura 8) situa o momento atual do projeto, indicado pela linha vertical, em relação às macroetapas planejadas para 2026.

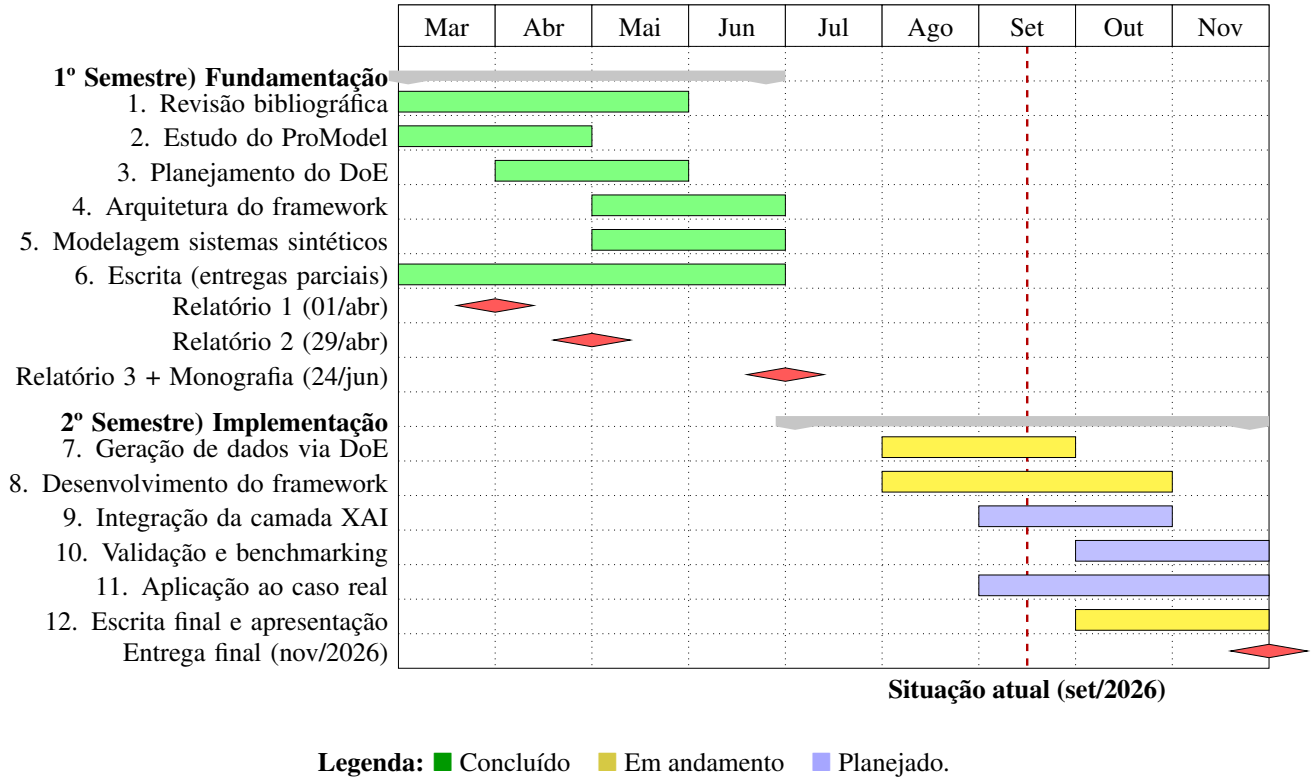



Figura 8: Diagrama de Gantt - Macroetapas do TCC 2026

# Apêndice B ORGANIZAÇÃO DO REPOSITÓRIO

O código-fonte do *framework* está disponível no ramo `main` do repositório

<https://github.com/MatzkinGurza/DES-Explainable_Surrogates-FW>

A estrutura de diretórios reflete a divisão em blocos descrita na Seção 4.1:

```
Código/
|-- configs/                        # cenários: parâmetros e KPIs
|-- notebooks/                      # roteiros de uso da cadeia
|-- runs/                           # artefatos por execução
|-- requirements.txt                # dependências e versões
|-- src/sed_meta/
|   |-- doe/                        # bloco de amostragem
|   |   |-- parameter_space.py      #   espaço e plano amostral
|   |   `-- replication.py          #   replicação e consolidação
|   |-- sim_interface/              # bloco de interface
|   |   `-- excel_bridge.py         #   planilha de cockpit
|   |-- preprocessing/              # bloco de pré-processamento
|   |   `-- pipeline.py             #   codificação e separação
|   |-- metamodels/                 # bloco de metamodelos
|   |   |-- baseline.py             #   modelos de referência
|   |   |-- tree_family.py          #   árvores de decisão
|   |   |-- gp_family.py            #   processos gaussianos
|   |   `-- neural_family.py        #   redes neurais
|   |-- evaluation/                 # avaliação estatística
|   |-- xai/                        # explicabilidade
|   |-- cycle/                      # ciclo de desenvolvimento
|   `-- logging_/                   # rastreabilidade
`-- tests/                          # verificação automatizada
```

Os diretórios `evaluation/`, `xai/`, `cycle/` e `logging_/` correspondem a blocos em construção.

Correspondência com as subseções do capítulo de implementação: `doe/` implementa as Seções 4.2 e 4.3; `sim_interface/` implementa a Seção 4.4; `preprocessing/` implementa a Seção 4.6; `metamodels/` implementa a Seção 4.7.

## Formato de troca entre blocos

Cada fronteira entre blocos é uma tabela com colunas nomeadas, o que concretiza o requisito de modularidade descrito na Seção 4.1 e permite que cada bloco seja executado isoladamente.

## Ambiente de referência

Python 3.12. As dependências e suas versões mínimas estão declaradas em `requirements.txt`. As bibliotecas centrais para o que foi descrito no capítulo de implemen-

Tabela 9: Formatos de troca entre os blocos do *framework*.

| **Bloco produtor** | **Formato produzido** | **Bloco consumidor** |
|---|---|---|
| Amostragem (DoE) | uma linha por ponto do plano, uma coluna por parâmetro | Replicação |
| Replicação | uma linha por execução, com identificador do ponto, índice da replicação e semente | Interface |
| Interface (escrita) | planilha de cockpit, com colunas de saída vazias | ProModel |
| Interface (leitura) | todas as execuções, com coluna de estado | Pré-processamento |
| Pré-processamento | matrizes de entrada e vetores de saída, já transformados | Metamodelos |

tação são NumPy e pandas para manipulação tabular, SMT e SciPy para amostragem, openpyxl para a planilha de troca, *scikit-learn* para pré-processamento e metamodelos, e XGBoost e hetGPy para variantes específicas de metamodelo.

# Apêndice C MÓDULOS E FUNÇÕES

Este apêndice descreve o que cada função pública de cada módulo faz. O código-fonte está disponível no repositório:

<https://github.com/MatzkinGurza/DES-Explainable_Surrogates-FW>

## C.1 Módulo de Design of Experiments (`doe/parameter_space.py`)

Tabela 10: Funções do módulo de Design of Experiments.

| Elemento | O que faz |
|---|---|
| `ParameterSpec` | Estrutura que descreve um parâmetro de entrada: nome, natureza (contínuo, inteiro, categórico, ordinal, binário), limites ou lista de níveis. É a declaração do espaço amostral que o restante da cadeia consome. |
| `build_design_space` | Traduz a lista de declarações para a representação de espaço de projeto da biblioteca SMT. |
| `generate_design` | Gera o plano amostral. Recebe as declarações, o número de pontos, a semente e o método de amostragem, e devolve uma tabela com uma linha por ponto e uma coluna por parâmetro, já com os tipos de dado corretos. É a função de entrada do bloco de amostragem. |

Internamente, `generate_design` separa o preenchimento do hipercubo unitário, que depende do método escolhido, da conversão para os domínios declarados, que é comum aos três métodos.

## C.2 Módulo de replicação (`doe/replication.py`)

Tabela 11: Funções do módulo de replicação.

| Elemento | O que faz |
|---|---|
| `ReplicationPolicy` | Estrutura que declara a política de replicação: número de replicações por ponto e semente base. |
| `build_execution_plan` | Expande o plano amostral em um plano de execução, repetindo cada ponto conforme a política e acrescentando identificador do ponto, índice da replicação e semente derivada dos dois. É a saída entregue ao bloco de interface. |
| `aggregate_replications` | Consolida as replicações de cada ponto, produzindo média, desvio amostral, contagem de replicações e erro padrão da média para cada indicador. |
| `compute_noise_ceiling` | Decompõe a variância observada em parcela entre pontos e parcela entre replicações do mesmo ponto, fornecendo a referência de variância explicável usada na avaliação. |

## C.3 Módulo de interface com o ProModel (`sim_interface/excel_bridge.py`)

Tabela 12: Funções do módulo de interface com o ProModel.

| Elemento | O que faz |
|---|---|
| `build_cockpit_table` | Acrescenta ao plano de execução as colunas de indicadores de saída, vazias, formando a tabela do cockpit. |
| `write_cockpit_excel` | Escreve a tabela em arquivo Excel, com a formatação de cabeçalho que distingue os blocos de entrada e saída e com a aba auxiliar que registra a natureza de cada coluna. |
| `read_cockpit_excel` | Lê o arquivo de volta, restaurando as naturezas das colunas a partir da aba auxiliar. |

| Elemento | O que faz |
|---|---|
| `extract_results` | Converte a planilha lida no formato consumido pelos módulos seguintes, classificando cada execução como completa, parcial, pendente ou inválida, sem remover linhas. |
| `select_complete_runs` | Seleciona as execuções completas, registrando o que ficou de fora. É o ponto único onde linhas deixam de seguir adiante na cadeia. |

## C.4 Módulo de pré-processamento (`preprocessing/pipeline.py`)

Tabela 13: Funções do módulo de pré-processamento.

| Elemento | O que faz |
|---|---|
| `TargetTransformSpec` | Estrutura que declara qual transformação aplicar a um indicador de saída: padronização, logaritmo seguido de padronização, ou nenhuma. |
| `build_input_transformer` | Monta o codificador dos parâmetros de entrada a partir das declarações do espaço amostral, sem consultar os dados. |
| `PreprocessingPipeline.split` | Separa treinamento e teste. Agrupa por identificador de ponto quando os dados estão na granularidade replicada. |
| `PreprocessingPipeline.fit` | Ajusta as transformações dos indicadores de saída, utilizando apenas o conjunto de treinamento. |
| `PreprocessingPipeline.transform_X` | Aplica a codificação aos parâmetros de entrada, devolvendo a matriz consumida pelos metamodelos. |
| `PreprocessingPipeline.transform_y` | Aplica a transformação declarada a um indicador de saída. |
| `PreprocessingPipeline.inverse_transform_y` | Desfaz a transformação, devolvendo previsões às unidades originais do indicador. |

| Elemento | O que faz |
|---|---|
| `PreprocessingPipeline.transform_sample_weight` | Converte o peso amostral produzido pela consolidação para a escala em que o indicador é ajustado. Aplicável apenas quando se opta pelo formato consolidado. |
| `PreprocessingPipeline.get_feature_names` | Devolve os nomes das colunas após a codificação, necessários para rotular as análises da etapa de explicabilidade. |
| `PreprocessingPipeline.save` e `load` | Serializam e recarregam o objeto ajustado, de modo que previsões futuras passem pela mesma transformação. |

## C.5 Módulos de metamodelos (`metamodels/`)

Os quatro módulos seguem a mesma convenção: para cada variante há uma função construtora, que devolve o modelo configurado com valores padrão adequados ao regime de dados do trabalho, e uma grade de hiperparâmetros sugerida. Todos os modelos seguem a interface de programação do scikit-learn, de modo que os procedimentos de validação cruzada e busca de hiperparâmetros daquela biblioteca funcionam sem adaptação.

Tabela 14: Variantes disponíveis em cada módulo de metamodelo.

| Módulo | Variantes disponíveis |
|---|---|
| `baseline.py` | preditor da média, regressão linear, regressão polinomial, regressão com regularização e regressão polinomial com regularização |
| `tree_family.py` | floresta aleatória, árvores extremamente aleatorizadas, reforço por gradiente e XGBoost |
| `gp_family.py` | processo gaussiano do scikit-learn, kriging da biblioteca SMT e processo gaussiano com ruído variável da biblioteca hetGPy |
| `neural_family.py` | perceptron multicamadas |

Cada módulo oferece também uma função de ajuste que repassa o peso amostral aos modelos que o aceitam e registra aviso para os que não o aceitam, de modo que a diferença de tratamento fique explícita no registro de execução.

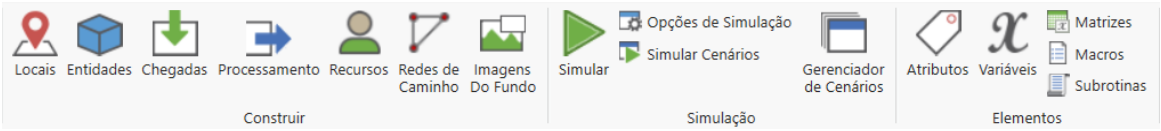
As duas variantes de processo gaussiano que não seguem a interface do scikit-learn são encapsuladas pelas classes `SMTKrigingRegressor` e `HetGPRegressor`, que traduzem as chamadas de ajuste e previsão. Esses adaptadores não contêm lógica de modelagem.

# Apêndice D FUNCIONAMENTO PROMODEL E MODELOS

Este apêndice descreve algumas características, ferramentas e funcionamento do ProModel, além de como foram consideradas algumas análises.

O ProModel foi introduzido na Seção 2.1 como *software* de Simulação de Eventos Discretos de interface orientada a processos e integração nativa com planilhas do *Excel* (BENSON, 1997). Esta subseção detalha o funcionamento prático do *software*, descrevendo as ferramentas empregadas na construção dos três modelos sintéticos. A Figura 9 apresenta a barra de ferramentas do ProModel utilizada para tal, organizada em três grupos funcionais: construção do modelo, simulação e elementos de programação.

Figura 9: Barra de ferramentas do ProModel.




**Fonte:** Captura de tela do *software* ProModel.

As ferramentas empregadas na construção dos três cenários sintéticos são:

- **Locais:** permitem a criação e o posicionamento dos elementos físicos do modelo, a distâncias arbitrárias entre si, além da definição da capacidade máxima de entidades presentes simultaneamente em cada local;
- **Entidades:** representam os objetos simulados, isto é, os itens que fluem pelo modelo e passam pelos processos ao longo da simulação;
- **Chegadas:** definem a entrada das entidades no sistema em locais específicos (no caso dos cenários desenvolvidos, no local de Entrada), permitindo configurar o número de ocorrências de chegada, a quantidade de entidades por chegada e a frequência entre chegadas consecutivas;
- **Processamento:** constitui o núcleo do modelo, no qual são definidas as sequências de deslocamento entre locais e os blocos de código responsáveis pela lógica condicional, pelos tempos de espera e pelas particularidades de cada processo, com o objetivo de aproximar o comportamento simulado do comportamento real do sistema;
- **Simular:** executa a simulação do modelo conforme as opções configuradas e viabiliza a exportação dos dados de saída e dos indicadores de desempenho resultantes;

- **Atributos:** variáveis associadas individualmente a cada entidade, carregadas por ela ao longo de toda a simulação, uma alteração feita no atributo de uma entidade específica não interfere nos atributos das demais. Nos modelos desenvolvidos, atributos são utilizados para determinar o caminho seguido por cada entidade, nos cenários com múltiplas opções de sequência, e para registrar os instantes de entrada e saída de cada entidade na simulação e em cada processo, dados que possibilitam o cálculo de indicadores derivados;

- **Variáveis:** diferem dos atributos por estarem associadas ao modelo como um todo, uma alteração realizada em qualquer processo é refletida globalmente. Essa característica torna as variáveis adequadas ao controle de sequências e lógicas do modelo, além do cálculo de indicadores agregados, como o número total de entradas e saídas por caminho, os tempos médios entre a entrada e cada processo, os tempos médios de cada processo, a quantidade de peças processadas em cada etapa e o tempo total de simulação. Com exceção do tempo total de simulação, todos os tempos são calculados como médias sobre o total de peças processadas, resultando em um valor único por indicador;

- **Matrizes:** sincronizam o modelo do ProModel com a planilha *Excel* do cockpit, os dados de entrada inseridos no cockpit alimentam diretamente os parâmetros da simulação e, ao término da execução, os indicadores de saída calculados pelo modelo são exportados automaticamente para células específicas da planilha, permitindo a visualização e a comparação entre cenários (Figura 10);

- **Subrotinas:** encapsulam trechos de código reutilizados repetidamente ao longo do modelo, evitando a duplicação de lógica. Nos modelos desenvolvidos, são empregadas três subrotinas: uma responsável pelo sorteio do caminho seguido pela entidade nos cenários com múltiplas opções de roteamento, definindo o atributo correspondente; uma segunda, acionada sempre que uma entidade conclui integralmente seu processamento, responsável por incrementar os acumuladores de tempo utilizados no cálculo das médias; e uma terceira, acionada ao término da simulação, responsável por calcular as médias finais de tempo e quantidade de peças processadas, obter o tempo total de simulação e organizar a exportação de cada indicador para a célula correspondente do cockpit.

A Figura 10 ilustra o resultado dessa sincronização para o Cenário 1: a área de Entrada reúne os parâmetros configuráveis da simulação (capacidades, quantidades e tempos de processo), enquanto a área de Saída reúne os indicadores de desempenho exportados ao final da execução.

Figura 10: Cockpit de entradas e saídas do Cenário 1.

| Entrada | |
|---|---|
| **Item** | **Valor** |
| Capacidade M1 | 1 |
| Quantidade chegada | 1 |
| Ocorrências chegada | 15 |
| Frequência chegada | 1 |
| Tempo processo 1 | 5 |
| Capacidade M2 | 1 |
| Capacidade M3 | 1 |
| Tempo processo 2 | 6 |
| Tempo processo 3 | 7 |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |
| | |

| Saída | |
|---|---|
| **Item** | **Valor** |
| Entradas | 15 |
| Prontas | 15 |
| Tempo_ate_P1 | 2356 |
| Tempo_ate_P2 | 2764 |
| Tempo_ate_P3 | 3180 |
| Tempo_P1 | 408 |
| Tempo_P2 | 416 |
| Tempo_P3 | 420 |
| Feito_P1 | 15 |
| Feito_P2 | 15 |
| Feito_P3 | 15 |
| Tempo_total | 3600 |
| Tempo_simul | 6960 |
| | 0 |
| | 0 |
| | 0 |
| | 0 |
| | 0 |
| | 0 |
| | 0 |

**Fonte:** Elaborado pelos autores.

Os três cenários sintéticos, apresentados conceitualmente na Seção 3.3.2, foram implementados seguindo a mesma proposta ali definida, e sua implementação mostrou-se viável para as condições de operação estabelecidas. À medida que a complexidade do cenário aumenta, cresce também o número de atributos e variáveis específicas por caminho ou por máquina, de modo a aproximar os indicadores de saída do comportamento real do sistema modelado.

**Referências**

ANGIONE, C.; SILVERMAN, E.; YANESKE, E. Using machine learning as a surrogate model for agent-based simulations. *PLOS ONE*, v. 17, n. 2, 2022.

BENSON, D. Simulation modeling and optimization using ProModel. *Proceedings of the 1997 Winter Simulation Conference*, p. 587–593, 1997. Disponível em: <https://www.informs-sim.org/wsc97papers/0587.PDF>. Acesso em: 28 mar. 2026. 12, 18, 68

BINOIS, M.; GRAMACY, R. B.; LUDKOVSKI, M. Replication or exploration? sequential design for stochastic simulation experiments. *Technometrics*, Taylor & Francis, v. 61, n. 1, p. 7–23, 2019. 18, 19, 20, 23, 28, 46

BODRIA, F. et al. Benchmarking and survey of explanation methods for black box models. *arXiv preprint arXiv:2102.13076*, 2021. Disponível em: <https://arxiv.org/abs/2102.13076>. Acesso em: 28 mar. 2026. 5, 6, 25, 26

BOUHLEL, M. A. et al. A python surrogate modeling framework with derivatives. *Advances in Engineering Software*, p. 102662, 2019. ISSN 0965-9978.

BREIMAN, L. Random forests. *Machine Learning*, Springer, v. 45, n. 1, p. 5–32, 2001. 5, 23, 27, 43, 44, 59

BROEKE, G. T. et al. The use of surrogate models to analyse agent-based models. *Journal of Artificial Societies and Social Simulation*, v. 24, n. 2, 2021.

CHAI, T.; DRAXLER, R. R. Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, Copernicus Publications, v. 7, n. 3, p. 1247–1250, 2014. 5, 6, 23, 28, 45, 46

FISHER, R. A. *The Design of Experiments*. [S.l.]: Oliver and Boyd, 1935. 18

FORRESTER, A. I. J.; SÓBESTER, A.; KEANE, A. J. *Engineering Design via Surrogate Modelling: A Practical Guide*. [S.l.]: John Wiley & Sons, 2008. ISBN 978-0-470-06068-1. 4, 5, 19, 20, 21, 22, 23, 43, 44

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016. Disponível em: <https://www.deeplearningbook.org>. 4, 23, 43, 44, 46

GROSS, D. et al. *Fundamentals of Queueing Theory*. 4. ed. [S.l.]: John Wiley & Sons, 2008. ISBN 978-0-471-79127-0. 39, 40

LAW, A. M. *Simulation Modeling and Analysis*. 4. ed. [S.l.]: McGraw-Hill, 2007. ISBN 978-0-07-329441-4. 42

MCKAY, M. D.; BECKMAN, R. J.; CONOVER, W. J. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, Taylor & Francis, v. 21, n. 2, p. 239–245, 1979. 4, 5, 19, 20

MOLNAR, C. *Interpretable Machine Learning*: A guide for making black box models explainable. 3. ed. [s.n.], 2025. ISBN 978-3-911578-03-5. Disponível em: <https://christophm.github.io/interpretable-ml-book>. 22, 23, 24, 25, 46

MONTEVECHI, J. A. B. et al. Ensemble-based infill search simulation optimization framework. In: *2023 Winter Simulation Conference (WSC)*. [S.l.: s.n.], 2023. p. 3059–3070. 4, 11, 18, 20, 21, 23, 28, 42

MONTGOMERY, D. C. *Design and Analysis of Experiments*. 9. ed. [S.l.]: John Wiley & Sons, 2017. 19

MUCHTAR, D. Bottleneck analysis in garment production process using simulation approach (promodel). *Jurnal Teknologika*, v. 15, n. 1, p. 643–651, 2025. ISSN 2715-4645. Disponível em: <https://jurnal.wastukancana.ac.id/index.php/teknologika/article/view/457>. Acesso em: 29 mar. 2026. 32

NAGAHISARCHOGHAEI, M. et al. An empirical survey on explainable AI technologies: Recent trends, use-cases, and categories from technical and application perspectives. *Electronics*, MDPI, v. 12, n. 5, p. 1092, 2023. 5, 6, 11, 22, 24, 25, 26, 27, 31

PALAR, P. et al. Multi-objective design space exploration using explainable surrogate models. *Structural and Multidisciplinary Optimization*, v. 67, n. 3, p. 38, 2024.

PATIL, A.; PATIL, M. A comprehensive review on explainable AI techniques, challenges, and future scope. In: ____. *Intelligent Computing and Networking*. [S.l.]: Springer Nature Singapore, 2023. (Lecture Notes in Networks and Systems, v. 699), p. 517–530. 11, 25

RIBEIRO, M. T.; SINGH, S.; GUESTRIN, C. *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*. 2016. Disponível em: <https://arxiv.org/abs/1602.04938>. 5, 11, 25, 26, 27

SALTELLI, A. et al. *Exploring Multi-Dimensional Spaces: A Comparison of Latin Hypercube and Quasi Monte Carlo Sampling Techniques*. 2015. 4, 20

SAVES, P. Interpretable and explainable surrogate modeling for simulations: A state-of-the-art survey and perspectives on explainable ai for decision-making. *arXiv preprint*, 2025. Disponível em: <https://www.academia.edu/145024476/Interpretable_and_Explainable_Surrogate_Modeling_for_Simulations_A_State_of_the_Art_Survey_and_Perspectives_on_Explainable_AI_for_Decision_Making>. Acesso em: 27 mar. 2026. 5, 6, 10, 11, 12, 13, 18, 21, 22, 23, 24, 25, 27

SAVES, P. et al. SMT 2.0: A surrogate modeling toolbox with a focus on hierarchical and mixed variables gaussian processes. *Advances in Engineering Sofware*, v. 188, p. 103571, 2024. 4, 5, 11, 14, 22, 23, 27

SAVES, P. et al. *Surrogate modeling and explainable artificial intelligence for complex systems: A workflow for automated simulation exploration*. 2025. ArXiv preprint.

SCHRIBER, T. J.; BRUNNER, D. T. Inside discrete-event simulation software: how it works and why it matters. *Proceedings of the 2003 Winter Simulation Conference*, p. 113–123, 2003. Disponível em: <https://www.informs-sim.org/wsc07papers/013.pdf>. Acesso em: 28 mar. 2026. 6, 12, 17, 38, 57

VIANA, F. A. C. A tutorial on latin hypercube design of experiments. *Quality and Reliability Engineering International*, Wiley, v. 32, n. 5, p. 1975–1985, 2016. 5, 20, 23

ZHANG, X. et al. Metamodel-interpreted data mining for stability and efficiency enhancement of multistage axial-flow compressors. *Journal of Turbomachinery*, v. 145, n. 4, p. 041001, 2023.